

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Big Data Analytics and Business Intelligence (CO4033)
CC01 - Group 06

Final Report

Big Data Analytics in Healthcare

Instructor:	Assoc. Prof. Vo Thi Ngoc Chau	, PhD
Students:	Le Trung Kien	2252394
	Nguyen Huu Cuong	2252098
	Ly Trieu Uy	2252889

HO CHI MINH CITY, January 21, 2026

Contents

Abstract	v
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Introduction to the MIMIC-IV dataset	3
1.4 Project Objectives	5
1.5 Technology Stack	6
1.5.1 Google BigQuery: Remote data source for large scale querying	7
1.5.2 Apache Kafka: Data streaming and ingestion	8
1.5.3 Spark: Structured stream processing	9
1.5.4 MinIO: S3-compatible data lake	10
1.5.5 PostgreSQL: Analytical Data Warehouse	10
1.5.6 Tableau: Business Intelligence and Visualization	11
1.5.7 Streamlit: Interactive Data Application	12
1.5.8 Python: Data Analytic and Machine Learning Layer	13
1.6 Project Scope	14
2 Methodology	16
2.1 Approach Overview	16
2.2 System Architecture	17
2.2.1 High-Level Architecture Description	17
2.2.2 System Architectural Design	19
3 Implementation	26
3.1 Implementation Environment and Tooling	26
3.2 Data Retrieval from Google BigQuery	28
3.3 Data Ingestion with Apache Kafka	29
3.4 Stream and Batch Processing with Apache Spark	30

3.4.1	Job 1: Kafka to Bronze Data Lake	30
3.4.2	Job 2: Bronze to Silver Transformation	31
3.4.3	Job 3: Load SCDs tables	32
3.4.4	Job 4: Load Facts into Analytical Warehouse	32
3.5	Data Lake Storage with MinIO	32
3.5.1	Data Lake Design Principles	33
3.5.2	Bronze Layer: Raw Streamed Data	33
3.5.3	Silver Layer: Cleaned and Integrated Fact Data	34
3.6	Analytical Warehouse Implementation with PostgreSQL	34
3.7	Analytics and Visualization Layer	36
3.7.1	ICU Overview	36
3.7.2	Trend Analysis	37
3.7.3	Correlation Analysis	38
3.7.4	Sepsis vs. Non-Sepsis Analysis in the Emergency Department	39
3.7.5	Predictive Analytics Module	41
3.7.6	Real-time Early Warning Module	47
4	Results and Discussion	50
4.1	Big Data Pipeline	50
4.1.1	Streaming Ingestion and Bronze Layer	50
4.1.2	Silver Layer Processing	51
4.1.3	Batch Integration and Warehouse Population	52
4.2	Analytical Outcomes	52
4.2.1	Module 1: Data OverView	53
4.2.2	Module 2 : Trend Analysis	54
4.2.3	Module 3: Correlation Analysis	55
4.2.4	Module 4 : Sepsis vs Non-Sepsis Analysis	56
4.2.5	Module 5: Real-time Early Warning System	57
4.2.6	Module 6: Predictive Clinical Decisanion Support	60
5	Conclusion	63
6	Appendix	65
6.1	Host System Specification	65
6.2	Full Project Source Code	65

List of Figures

1.1	MIMIC-IV's modular structure [6].	5
1.2	Google BigQuery	7
1.3	Apache Kafka	8
1.4	Apache Spark	9
1.5	MinIO	10
1.6	PostgreSQL	10
1.7	Tableau for Visual Analytics	11
1.8	Streamlit Framework	12
1.9	Python Data Science Ecosystem	13
2.1	Abstract data flow of the system	17
2.2	System architecture design (1)	20
2.3	System architecture design (2)	21
2.4	System architecture design (3)	22
2.5	System architecture design (4)	23
2.6	System architecture design (5)	24
2.7	System architecture design (complete)	25
3.1	Data warehouse schema diagram	35
3.2	Average Readmission Risk Scores aggregated by key patient features. Patients with > 2 previous admissions and Long LoS (>7 days) show the highest risk probability.	44
3.3	Top 15 Diagnosis Categories by Readmission Rate. High rates are observed in mental health (F32, F10) and chronic conditions.	45
3.4	Precision-Recall Curve for Readmission Model. The optimal threshold was selected to prioritize Recall.	46
3.5	Distribution of Real-time Alerts by Vital Sign. The system aggregates high-frequency streams to highlight the most prevalent physiological instabilities. . . .	49
3.6	Patient List	49
4.1	Bronze tier data lake	51
4.2	Resulted analytical warehouse	52
4.3	Overview of ICU utilization. The left panel illustrates ICU stays by admission date, showing temporal trends in ICU admissions over time. The right panel depicts ICU stays by care unit, highlighting the distribution of ICU utilization across different types of intensive care units.	53

4.4	Trend analysis of ICU utilization. The left panel shows the top ten ICD diagnosis codes by ICU stay count, highlighting the most frequent conditions requiring intensive care. The right panel presents the gender distribution of ICU stays, illustrating the proportion of male and female patients admitted to the ICU. . . .	54
4.5	Correlation analysis of physiological and clinical indicators. The left panel shows the relationship between average heart rate and average oxygen saturation (SpO ₂) in ICU patients, aggregated hourly within the same ICU stay. The right panel illustrates the correlation between triage acuity level and heart rate in the emergency department, where higher acuity values indicate greater severity.	55
4.6	Comparison of sepsis and non-sepsis ICU patients. The figure shows the distribution of ICU patients by sepsis status, highlighting a substantial imbalance between non-sepsis and sepsis cases.	56
4.7	The Real-time Early Warning Dashboard displaying active alerts and patient status.	57
4.8	Manual Prediction Interface for assessing individual patient risk across multiple outcomes.	60

List of Tables

1.1	Overall architectural view of technologies	6
3.1	Containerized implementation environment overview	27
3.2	Selected MIMIC-IV schemas and tables used in the system	28
3.3	Performance trade-off: XGBoost sacrifices overall accuracy to triple the detection rate of critical Long-Stay patients.	43
3.4	Top 10 Most Important Features for Mortality Prediction	47
6.1	System Hardware Specification	65

Abstract

Modern healthcare environments generate massive quantities of complex data characterized by high volume and high velocity, presenting significant challenges for traditional storage and processing systems. This project designs and simulates an end-to-end big data pipeline capable of managing these dimensions using the MIMIC-IV clinical dataset as a realistic foundation. The implemented architecture adopts a modular, layered approach utilizing Dockerized open-source technologies, including Apache Kafka for high-velocity streaming ingestion and Apache Spark for unified structured streaming and batch processing.

The system methodology follows a progressive data flow where raw event data is ingested into a tiered data lake (MinIO) comprising Bronze and Silver layers, ensuring durability and traceability before being integrated into a PostgreSQL analytical warehouse. To demonstrate the system's value, the project implements a robust analytics and visualization layer using Tableau and Streamlit. Results include descriptive analytics regarding ICU admission trends and physiological correlations, as well as advanced predictive modules using XGBoost to forecast patient mortality, readmission risk, and length of stay. Additionally, a real-time Early Warning System was developed to monitor vital sign streams and detect immediate patient instability, validating the pipeline's ability to support both retrospective business intelligence and proactive clinical decision support.

Chapter 1

Introduction

1.1 Background

Big data refers to datasets that are too large or complex for traditional data-processing applications to handle effectively. It is commonly described by five key characteristics, known as the 5Vs: volume, velocity, variety, veracity, and value. Volume represents the sheer scale of data, often in the range of terabytes or petabytes, which challenges storage and analysis capacities. Velocity denotes the high speed at which data is generated, ingested, and processed, requiring near-real-time handling to derive timely insights. These aspects are particularly critical in domains like healthcare, where big data enables improved patient outcomes through predictive analytics and personalized care [1].

In healthcare applications, big data often involves integrating sensor data from monitoring devices—such as vital signs in intensive care units (ICUs)—with biographic details (e.g., patient demographics) and bibliographic records (e.g., medical histories and lab results). For instance, continuous physiological signals from ICU sensors generate high-velocity streams that must be processed rapidly to support real-time decision-making, while the accumulated volume from multiple patients over time demands scalable storage solutions. This combination facilitates analytics for trends in patient conditions, resource allocation, and early intervention [2].

Healthcare is a representative domain where Big Data challenges are both prominent and consequential. Modern healthcare systems generate data from multiple sources, including electronic health records (EHRs), bedside monitor-

ing devices, laboratory systems, and clinical documentation. In particular, sensor and monitoring devices in intensive care units (ICUs) continuously produce time-stamped physiological signals such as heart rate, blood pressure, oxygen saturation, and respiratory rate. These high-frequency measurements are often combined with static or slowly changing data, such as patient demographics, admission records, diagnoses, and procedures, to support clinical decision-making and downstream analytics.

From a theoretical perspective of big data,

- *Volume* refers to data sizes that exceed the capacity of traditional single-node storage and processing systems. As datasets grow to tens or hundreds of gigabytes, and potentially to terabytes or petabytes in large institutions, issues such as storage scalability, distributed data access, and parallel computation become critical. Conventional relational database systems and batch-oriented workflows are often insufficient to handle such scale efficiently without architectural changes.
- *Velocity*, on the other hand, describes the speed at which data is generated, ingested, and processed. In healthcare monitoring scenarios, data may arrive continuously and at high frequency, requiring near real-time processing to avoid backlogs, data loss, or delayed insights. High velocity introduces challenges in stream ingestion, message buffering, fault tolerance, and low-latency processing. Systems designed solely for offline batch analytics struggle to cope with these requirements.

This project focuses on Volume and Velocity as the primary dimensions of interest. They form the core motivation for adopting distributed storage systems, stream processing frameworks, and scalable data pipelines. The discussion in this chapter serves as a conceptual prelude to the dataset and system design choices presented later, particularly in relation to the MIMIC-IV dataset.

1.2 Problem Statement

Despite significant advances in big data technologies, managing healthcare data at scale remains a complex problem. Large volumes of historical patient data can quickly exceed storage and processing limits if not properly distributed, leading to

long query times and limited analytical flexibility. At the same time, high-velocity data streams—such as simulated sensor or event-based data—require timely ingestion and processing to ensure that downstream analytics remain relevant and responsive.

In healthcare-related scenarios, these challenges manifest in several ways. Storage systems may struggle to accommodate continuous data growth, while traditional relational databases are often not optimized for large-scale ingestion or parallel processing. In streaming contexts, delays in data processing can result in outdated insights, reduced situational awareness, or missed opportunities for early intervention. These issues highlight the need for end-to-end data pipelines that can handle both batch and streaming workloads efficiently.

The core problem addressed in this project is therefore how to design and simulate a big data pipeline that effectively manages large data volumes and high data velocity, using a realistic healthcare dataset. The challenge is not only technical ingestion and storage, but also enabling downstream analytics and business intelligence in a timely and scalable manner. This project investigates how modern big data components—such as distributed storage, stream processing, and analytical layers—can be integrated into a coherent workflow that mitigates storage overflow and processing latency while remaining suitable for analytical and BI use cases.

1.3 Introduction to the MIMIC-IV dataset

The MIMIC-IV (Medical Information Mart for Intensive Care IV) dataset is a large, openly available clinical database developed by the MIT Laboratory for Computational Physiology in collaboration with Beth Israel Deaconess Medical Center. It contains de-identified health data associated with patients admitted to intensive care units (ICUs) and emergency departments, covering a time span from 2008 to 2019 [3].

From a big data perspective, MIMIC-IV is particularly valuable because it combines scale, complexity, and realism. The full dataset is approximately 100 GB in size when uncompressed, consisting of dozens of interrelated tables stored in a relational format. These tables represent different aspects of patient care, including

demographics, admissions, diagnoses, procedures, laboratory results, medications, and clinical events. Core tables collectively capture both longitudinal patient histories and high-frequency clinical observations.

The dataset naturally exhibits high data volume due to the large number of patients and the fine granularity of recorded events. For example, tables such as `chartevents` contain more than 300 millions rows [4], recording bedside measurements at frequent time intervals. Processing or querying such tables using traditional methods can be inefficient, making MIMIC-IV well suited for investigating and evaluating the "big data methodology", like distributed storage and parallel processing solutions.

In addition to volume, MIMIC-IV also lends itself to velocity-oriented simulations. Although the dataset itself is static, many of its tables represent data that was originally generated in near real time, such as vital signs and monitoring events. These characteristics allow subsets of the data to be replayed or streamed, effectively simulating real-time ingestion scenarios similar to live sensor feeds in clinical environments. This makes the dataset suitable for experimenting with streaming platforms and structured streaming engines without requiring access to real hospital systems.

Another important aspect of MIMIC-IV is its structured and well-documented schema. The database design follows clear relational principles, divided into modules, with consistent identifiers and detailed metadata. Modules can be linked by identifiers including `subject_id`, `hadm_id`, and deidentified date and time, [Figure 1.1](#) shows the MIMIC-IV's modular structure. This structure enables the dataset to be ingested into data lakes, analytical warehouses, with minimal ambiguity, while still being complex enough to reflect real-world healthcare data challenges [5].

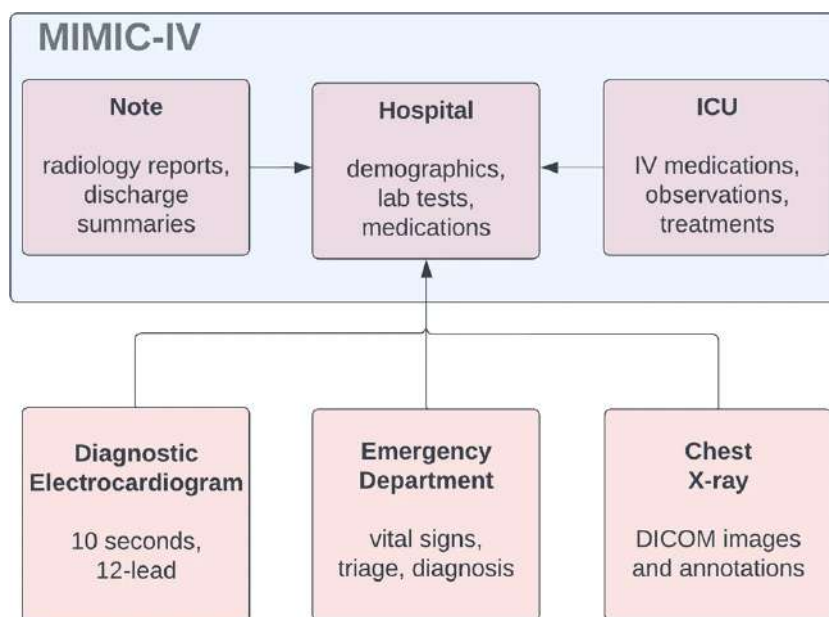


Figure 1.1: MIMIC-IV's modular structure [6].

In this project, MIMIC-IV serves as the primary data source for simulating big data solutions focused on Volume and Velocity. Its size stresses storage and batch processing components, while its event-based nature supports streaming simulations. At the same time, the dataset's clinical relevance allows downstream analytics and business intelligence tasks to be framed in a realistic healthcare context, rather than as abstract technical exercises.

1.4 Project Objectives

The primary objective of this project is to design and simulate an end-to-end big data pipeline capable of handling large data volumes and high data velocity in a healthcare context. Using the MIMIC-IV dataset as a realistic foundation, the project demonstrates how modern big data technologies can be integrated to support scalable ingestion, processing, storage, and analytics. Specifically, the project aims to:

- Design and implement a data pipeline capable of storing and processing large volumes of healthcare data.
- Simulate high-velocity data streams and demonstrate real-time or near-real-time ingestion.

- Integrate batch and streaming components into a coherent end-to-end architecture, from data ingestion to analysis.
- Beyond ingestion and processing, the project explicitly focuses on analytics and business intelligence (BI). An important objective is to demonstrate how data, processed with the above facilities, can be consumed by analytical tools to produce insights.

Finally, the project aims to provide a practical learning-oriented implementation rather than a purely theoretical system. The objective is not to build a production-grade healthcare platform, but to create a coherent and reproducible workflow that illustrates key big data concepts. By covering ingestion, streaming, storage, processing, and analytics in a single pipeline.

1.5 Technology Stack

This project adopts a layered big data architecture, where each technology addresses a specific responsibility in the end-to-end workflow, from ingestion to analytics and business intelligence. The stack is intentionally composed of widely used, industry-relevant tools to reflect realistic big data system design, specific technologies are presented in [1.1](#).

Table 1.1: Overall architectural view of technologies

Layer	Purpose	Technology
Cloud data storage	Remote, cloud-based data source, from which data is extracted	Google BigQuery
Ingestion & streaming	High-velocity data ingestion and buffering	Apache Kafka
Processing	Batch and stream processing at scale	Apache Spark
Storage	Scalable raw and curated data storage	MinIO/S3, PostgreSQL

Continued on next page

Table 1.1: Overall architectural view of technologies (continued)

Layer	Purpose	Technology
Analytics & BI	Querying, analysis, and visualization	Tableau, Streamlit, Python

The remainder of this section we will present each individual technologies, giving an abstract view of their properties and how they fit into our project. While further architectural details will be discussed in [2.2](#).

1.5.1 Google BigQuery: Remote data source for large scale querying

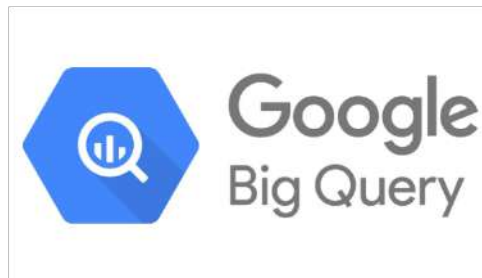


Figure 1.2: Google BigQuery

Google BigQuery is used as a cloud-based engine for large-scale data querying. They employ a serverless architecture with columnar storage and massively parallel execution. Users can query large datasets using SQL without managing infrastructure, making it well suited for exploratory and ad-hoc analytics on very large tables. It offers [\[7\]](#):

- Column-oriented storage optimized for analytical queries
- Automatic scaling and parallelism
- Efficient handling of very large datasets

In the context of this project, BigQuery is used to explore and analyze large portions of the MIMIC-IV dataset, particularly during inspection and validation

phases. It demonstrates how data Volume challenges can be addressed using cloud-native analytical warehouses, complementing on-premise or self-managed components.

1.5.2 Apache Kafka: Data streaming and ingestion



Figure 1.3: Apache Kafka

Apache Kafka is used as the event ingestion and streaming backbone of the system.

Kafka is designed around an append-only distributed log, which allows it to handle extremely high ingestion rates while maintaining durability and ordering guarantees. Unlike traditional message queues, Kafka retains data for a configurable period, enabling multiple consumers to read the same stream independently and at different speeds. Some of its capabilities includes [8]:

- High-throughput ingestion suitable for simulating continuous sensor and event data
- Partitioned topics enabling horizontal scalability
- Decoupling of producers and consumers, reducing system bottlenecks

In this project, Kafka enables the simulation of high-velocity healthcare data streams. We will create a producer replaying event-based MIMIC-IV for Kafka to record as ordered streams. This closely mirrors real ICU monitoring scenarios, where data arrives continuously and must be processed incrementally rather than in large batches. Kafka's design makes it particularly suitable.

1.5.3 Spark: Structured stream processing



Figure 1.4: Apache Spark

Apache Spark Structured Streaming serves as the core processing engine for both batch and streaming workloads.

Structured Streaming is built on Spark SQL and uses a micro-batch or continuous processing model to treat streaming data as an unbounded table. This abstraction allows developers to apply the same transformation logic to historical datasets and live streams. Spark offers the following characteristics [9]:

- Spark offers a unified batch and streaming APIs based on DataFrame.
- Fault tolerance and exactly-once processing guarantees. Which is suitable for Kafka. Apache Kafka is "at least once" guaranteed by default [10].
- Spark is also designed to scales across distributed compute clusters.

For this project, Spark Structured Streaming directly addresses the intersection of Volume and Velocity. Large historical tables from MIMIC-IV are processed in batch mode, while simulated real-time streams from Kafka are processed incrementally using the same APIs. This reduces system complexity and highlights how modern big data frameworks blur the line between batch and stream processing.

1.5.4 MinIO: S3-compatible data lake



Figure 1.5: MinIO

MinIO is used as the data lake storage layer. It implements an object storage model compatible with Amazon S3, which has become the de facto standard for data lakes. Object storage is optimized for large, immutable files and supports parallel access patterns common in big data processing. MinIO features offer the standards of data lakes [11]:

- Scalable storage for large datasets
- Schema-on-read flexibility
- Seamless integration with Spark and other analytics engines

In this project, MinIO stores raw and cleaned or enriched datasets generated from streaming pipelines. This supports a lakehouse-style architecture, where the data lake acts as the central persistence layer while remaining decoupled from compute.

1.5.5 PostgreSQL: Analytical Data Warehouse



Figure 1.6: PostgreSQL

While PostgreSQL is not designed for raw big data ingestion, it excels at managing curated, relational datasets with strong consistency and rich SQL support. It is widely supported by BI tools and analytics frameworks. Some of its capabilities includes[12]:

- Robust relational modeling and indexing
- Strong SQL compliance
- Compatibility with analytics and visualization tools

In this project, PostgreSQL stores analytics-ready datasets derived from the silver data lake. It acts as a bridge between big data processing systems and BI tools, illustrating a common architectural pattern where large-scale processing is separated from interactive analytical querying

1.5.6 Tableau: Business Intelligence and Visualization

Tableau serves as the Business Intelligence (BI) and visualization layer of the project. While big data systems excel at processing and storage, raw data remains unintelligible to end-users without effective visual representation. Tableau provides an intuitive interface to create interactive dashboards and reports [13].



Figure 1.7: Tableau for Visual Analytics

Its key capabilities utilized in this project include:

- **Direct connectivity:** Seamless integration with PostgreSQL and other SQL-based warehouses to fetch curated data.
- **Interactive visualization:** Drag-and-drop functionality to create dynamic charts, heatmaps, and trend lines without writing complex code.

- **Dashboarding:** Aggregating multiple views into coherent dashboards for monitoring key performance indicators (KPIs) [13].

In this architecture, Tableau connects to the PostgreSQL Analytical Warehouse. It is used to visualize the "Value" aspect of the 5Vs, transforming processed MIMIC-IV data—such as patient demographic distributions, ICU occupancy trends, and aggregated vital signs—into actionable clinical insights. This demonstrates how the pipeline supports downstream decision-making.

1.5.7 Streamlit: Interactive Data Application

Streamlit is an open-source Python framework used to create custom web applications for data science and machine learning. Unlike traditional BI tools like Tableau, which focus on static or historical data visualization, Streamlit allows for the rapid development of interactive, script-based applications [14].



Figure 1.8: Streamlit Framework

It is characterized by:

- **Python-centric design:** Allows developers to turn data scripts into shareable web apps in minutes using pure Python [14].
- **Interactivity:** Supports widgets (sliders, inputs, buttons) that can trigger Python code execution in real-time.
- **ML Integration:** Natively supports displaying outputs from machine learning models, making it ideal for demonstrating predictive capabilities.

For this project, Streamlit serves as the interface for specific analytical tasks that require user interaction or real-time simulation feedback. It acts as a frontend

for the predictive models (implemented in Python) and as a monitoring tool for the ingestion pipeline, bridging the gap between the backend processing system and the end-user.

1.5.8 Python: Data Analytic and Machine Learning Layer

Python functions as the "glue" language and the core analytical engine for the project. Beyond its role in orchestration (e.g., writing Kafka producers or PySpark scripts), Python is utilized here specifically for its rich ecosystem of data science libraries [15].



Figure 1.9: Python Data Science Ecosystem

Key libraries employed in the analytic layer include:

- **Pandas:** For data manipulation and analysis of structured data retrieved from the warehouse or intermediate CSV files **pandas'2024**.
- **Scikit-learn / XGBoost:** For implementing predictive analytics and machine learning algorithms (e.g., Random Forest, Gradient Boosting) **scikit'learn'2024**, **xgboost'2024**.
- **Matplotlib / Seaborn:** For exploratory data analysis (EDA) and generating static plots for reports.

In the context of the MIMIC-IV dataset, this layer is responsible for advanced analytics tasks that go beyond SQL aggregation, such as predicting ICU length of

stay, mortality risk, or readmission probability. Python consumes the clean data prepared by the Spark pipeline and hosted in PostgreSQL to generate high-value predictive insights, which are then exposed via the Streamlit application.

1.6 Project Scope

This project is scoped to provide a focused and practical exploration of big data system design, with particular emphasis on the challenges of data Volume and Velocity in a healthcare context. The scope is intentionally constrained to ensure that the technical implementation remains coherent and that the discussion stays aligned with the project objectives.

The MIMIC-IV dataset is used as the sole data source throughout the project. Both batch-oriented and streaming-style workloads are derived from this dataset, allowing historical clinical data to be processed at scale while also simulating real-time data generation scenarios. This approach ensures consistency across the pipeline and enables meaningful comparison between batch and streaming processing strategies using the same underlying data foundation [5].

From a big data perspective, the project explicitly focuses on technologies of the dimensions of Volume and Velocity. Volume is addressed through the ingestion, storage, and processing of large clinical tables that exceed the capacity of traditional systems. Velocity is addressed through simulated streaming ingestion and near-real-time processing, which reflect the continuous data flows commonly found in intensive care and monitoring environments. Other big data dimensions, such as Variety, Veracity, and Value, are acknowledged but not treated as areas of investigation.

The project covers the complete data lifecycle, from ingestion to analytics. This includes batch ingestion of large datasets, streaming ingestion using an event-driven platform, distributed processing, storage in a tiered data lake, population of an analytical warehouse, and consumption by analytics and business intelligence tools. The scope deliberately includes downstream analytics to demonstrate how engineering decisions at the data pipeline level directly affect analytical performance and usability.



This project is primarily concerned with the investigation, setup, and basic usage of big data streaming and processing technologies, rather than with building a fully optimized or production-scale distributed system. Due to practical limitations in accessing large-scale distributed computing facilities, the project focuses on understanding how these technologies are configured, integrated, and used in practice, and how they behave when applied to realistic big data workloads.

Chapter 2

Methodology

2.1 Approach Overview

The methodology of this project is designed as an exploratory and implementation-focused study of big data streaming and processing technologies. Rather than aiming to build a fully optimized or production-grade distributed system, the project investigates how key big data components are set up, connected, and used together to handle data Volume and Velocity under realistic but constrained conditions.

The core methodological idea is to construct a simulated end-to-end data pipeline that mirrors common big data architectures used in practice. This pipeline spans data retrieval, ingestion, streaming and batch processing, storage, and analytics. Each stage is implemented using widely adopted technologies to ensure that the experience and observations remain relevant to real-world systems, even though the execution environment is limited to local or small-scale resources.

A central design principle of the methodology is modularity. Each component of the pipeline—such as Kafka for ingestion, Spark for processing, object storage for the data lake, and relational storage for analytics—is treated as an independent module with a clearly defined responsibility. This separation allows the behavior and role of each technology to be studied in isolation, while still observing how they interact as part of a larger system.

Due to the lack of access to large-scale distributed computing infrastructure, the methodology deliberately prioritizes functional correctness, data flow valida-

tion, and architectural understanding over raw performance benchmarking. For example, while the pipeline is capable of processing large datasets and high ingestion rates, the total volume of data persisted in storage is capped at a manageable scale. This allows the project to focus on configuration, orchestration, and data handling logic under hardware constraints.

Another key methodological choice is the simulation of Velocity using historical data. Although the MIMIC-IV dataset is static, subsets of event-based data are replayed as streams to emulate real-time data generation.

Finally, the methodology explicitly includes business intelligence as part of the evaluation. Processed data is exposed to analytical tools to verify that it is usable, consistent, and suitable for analysis. This reinforces the idea that big data systems should be assessed not only by their ability to ingest and process data, but also by how effectively they support analytical and decision-oriented use cases.

2.2 System Architecture

2.2.1 High-Level Architecture Description

The system architecture of this project is designed as a layered, end-to-end data pipeline that reflects common patterns in modern big data systems. [Figure 2.1](#) shows the flow of data in the system at a high level, the pipeline follows a clear flow. Each stage is implemented using a dedicated technology that addresses a specific functional requirement, which is also match with the overview of the technology stack presented in [Table 1.1](#).

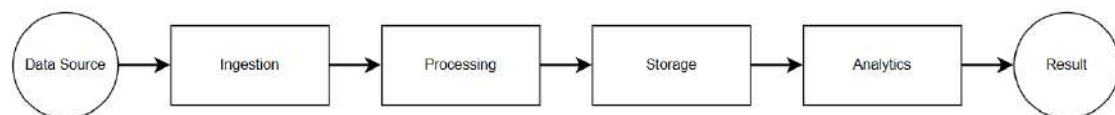


Figure 2.1: Abstract data flow of the system

The pipeline begins with the data source, the MIMIC-IV dataset accessed through Google BigQuery. BigQuery serves as the entry point for large-volume

historical data, enabling selective querying and extraction without requiring the full dataset to be stored locally. The dataset is conceptually divided into two categories: high-velocity fact-like data, such as event and measurement records, and more slowly changing dimension-like data, such as patient demographics and admission information. This separation directly influences how data is ingested and processed downstream.

For data ingestion, Apache Kafka is used to simulate high-velocity data streams. Event-based fact data extracted from MIMIC-IV is published to Kafka topics at controlled rates, emulating continuous data generation. Kafka acts as a buffering and decoupling layer, isolating data producers from consumers and allowing downstream systems to process data independently of ingestion speed. This design is particularly important for handling velocity-oriented workloads, where ingestion rates may fluctuate over time.

Data processing is performed using Apache Spark, with a clear distinction between streaming and batch workflows.

- Streaming jobs subscribe to Kafka topics and focus primarily on ingesting, validating, and minimally transforming high-velocity fact data. These jobs are designed to operate continuously, processing incoming records incrementally and writing the results directly to the data lake.
- In contrast, batch-oriented Spark jobs operate on stored data. They read previously ingested streaming outputs from the data lake, combine them with slowly changing dimension data retrieved from the source, and perform more complex transformations and integrations. The output of these batch jobs is analytics-ready data suitable for downstream consumption.

The storage layer is implemented using a data lake architecture based on MinIO. Streaming jobs write raw or lightly processed data into the Bronze layer, preserving the original structure and arrival order of the data. another job then produce cleaned, standardized, and integrated datasets in the Silver layer.

For analytical storage, selected Silver-layer datasets are loaded into a PostgreSQL analytical warehouse. This layer provides a structured, relational view of

the data that is optimized for querying, aggregation, and integration with analytics tools. By separating large-scale storage from analytics-serving storage, the system avoids overloading the warehouse with raw or high-frequency data.

Finally, the analytics layer consists of Tableau and Streamlit applications that consume data from the analytical warehouse and, where appropriate, directly from curated lake datasets. Tableau is used for structured dashboards and business intelligence reporting, while Streamlit supports interactive exploration and custom analytical views. This dual approach demonstrates how the same processed data can support different analytical use cases.

Design justification

The interaction between batch-oriented and streaming-oriented workflows is a central architectural feature. High-velocity fact data is handled independently by streaming pipelines, ensuring that continuous ingestion does not block or slow down batch processing. Batch workflows operate on stable snapshots of stored data, combining streamed facts with dimension data to produce consistent analytical datasets. This separation aligns with established big data design patterns and simplifies both development and debugging.

A layered architecture is chosen over a monolithic design to improve clarity, flexibility, and fault isolation. In a monolithic system, ingestion, processing, storage, and analytics are tightly coupled, making the system harder to understand and modify. By contrast, the layered approach used in this project allows each component to evolve independently, supports experimentation with different processing strategies, and reflects the structure of real-world big data platforms. This design choice is particularly suitable for an investigative project, where understanding component roles and data flow is more important than minimizing system complexity at all costs.

2.2.2 System Architectural Design

In this section, we progressively introduce the architectural diagram for this project, following the flow of data from its origin to final analytical consumption. This step-by-step construction helps clarify the role of each component and how the overall

system evolves from a simple data source into a complete big data pipeline. The presentation aligns with the layered architecture described earlier, while grouping processing and storage elements where they are tightly interdependent.

Data Source

Figure 2.2 marks the starting point of the architecture is Google BigQuery, which acts as an external data source. Conceptually, BigQuery represents hospital information systems that generate and store large volumes of clinical data. Although it is not part of the core system being built, it provides standardized APIs that allow large datasets to be queried and retrieved efficiently. This abstraction allows the project to focus on downstream big data processing without requiring direct access to hospital infrastructure.

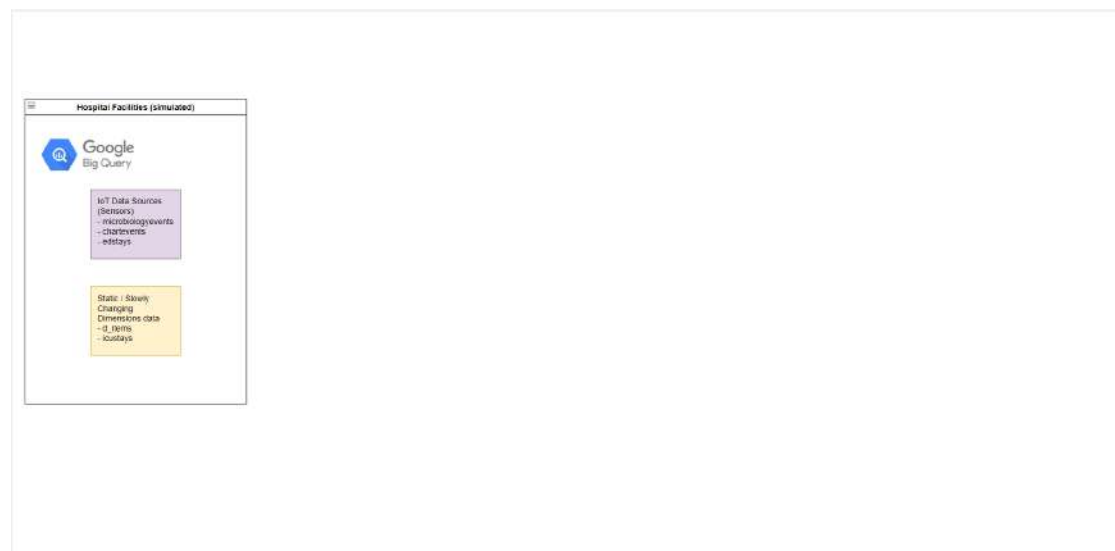


Figure 2.2: System architecture design (1)

Ingestion

The flow progresses to Figure 2.3. The next architectural addition is Apache Kafka, introduced to handle the continuous flow of incoming data. Once data is retrieved from BigQuery, selected subsets—particularly high-velocity fact data—are streamed into the system via Kafka. At this stage of the diagram, Kafka represents the ingestion and buffering layer. Its role is to absorb incoming data streams, preserve ordering within partitions, and decouple data producers from downstream

consumers. Kafka is not intended for long-term storage; instead, it acts as a transient layer that smooths ingestion rates and protects downstream systems from spikes in data velocity.

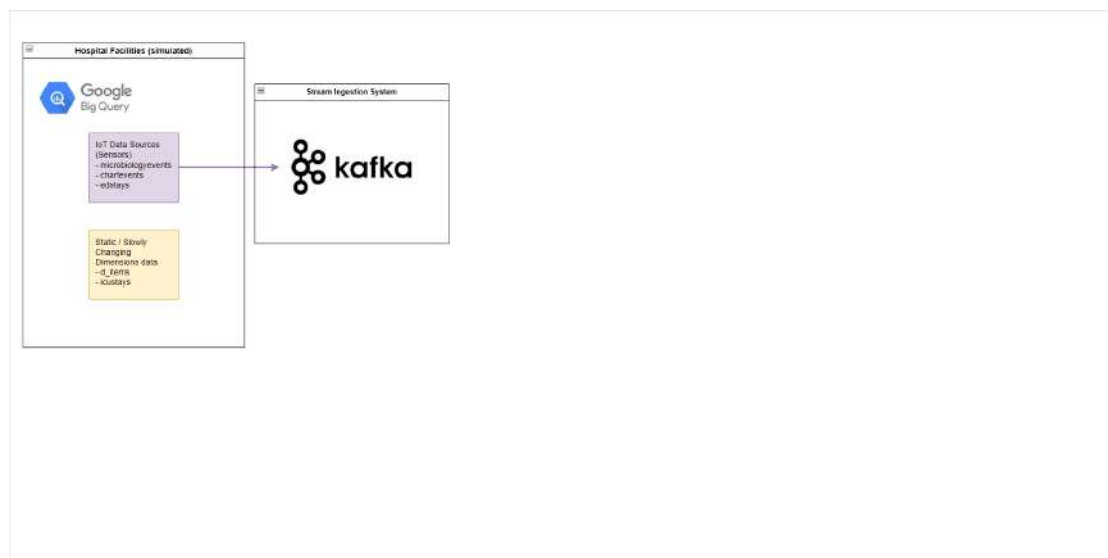


Figure 2.3: System architecture design (2)

Processing and Storage

As the architecture evolves in [Figure 2.4](#), it becomes necessary to move data out of Kafka and into persistent storage. This introduces Apache Spark jobs and the data lake as a combined processing-and-storage layer. The first Spark streaming job (Job 1) subscribes to Kafka topics and focuses on durability rather than transformation. Its primary responsibility is to write incoming data to the Bronze tier of the data lake, preserving the original structure, schema, and arrival order of the records. At this point in the diagram, the system ensures that streamed data is no longer ephemeral and can be replayed or reprocessed if needed.

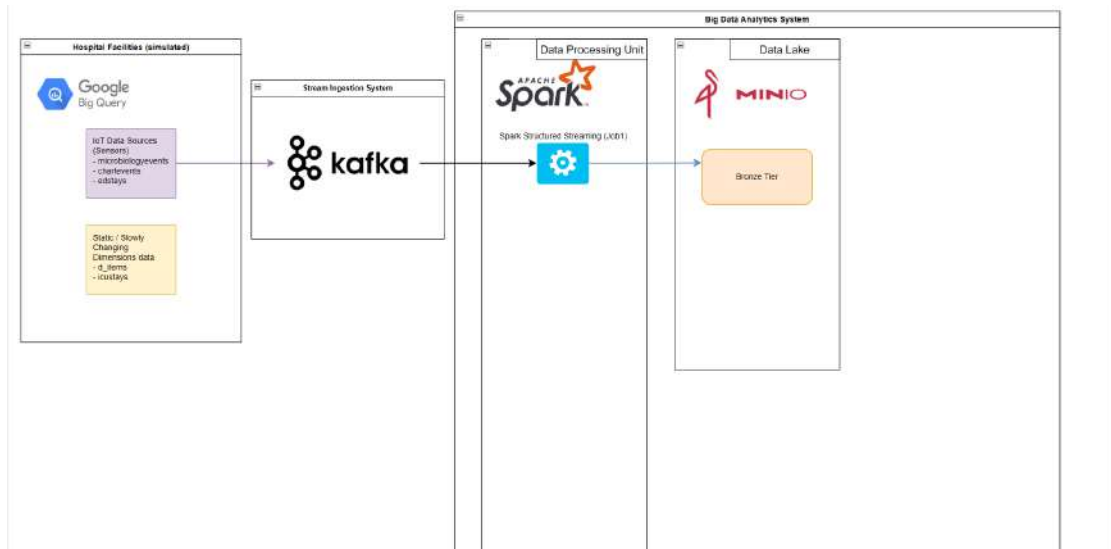


Figure 2.4: System architecture design (3)

However, raw data stored in the Bronze tier has limited direct analytical value. This motivates the next architectural step in [Figure 2.5](#), where a second Spark job (Job 2) reads from the Bronze layer and performs parsing, cleaning, and basic standardization. The output of this job is written to the Silver tier of the data lake. The Silver tier contains large, relatively intact datasets that are cleaner and more consistent than raw inputs, while still remaining detailed and abundant. This layer is intended for exploratory analysis and downstream processing rather than for end-user querying.

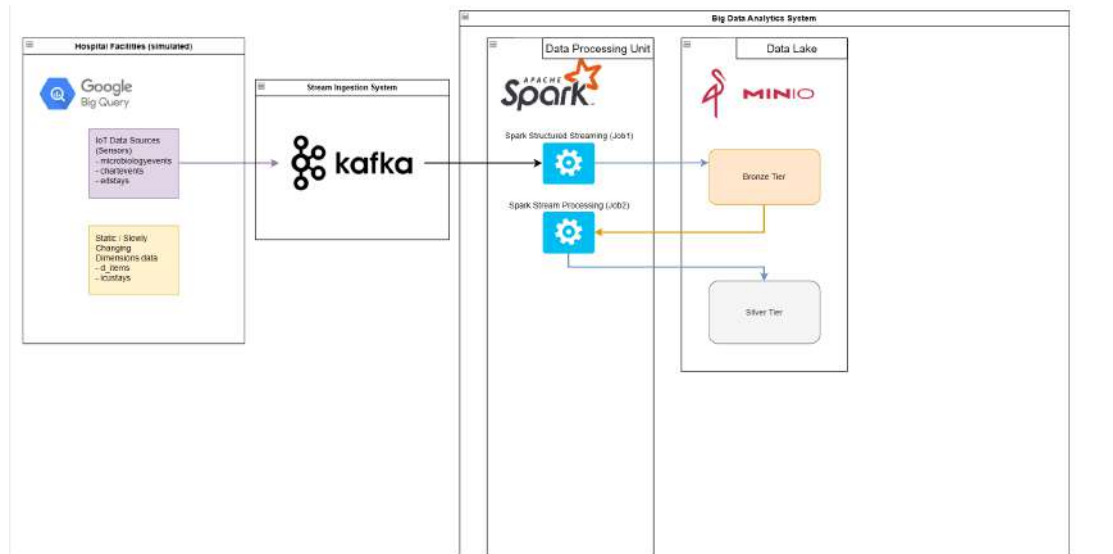


Figure 2.5: System architecture design (4)

Although the Silver tier contains cleaned fact data, meaningful analysis often requires additional context provided by slowly changing dimension (SCD) tables, such as patient demographics or admission metadata. A third Spark job (Job 3) integrates streamed fact data from the Silver layer with dimension data sourced from batch inputs. At this point, the architectural diagram highlights a limitation: while the data lake is effective for large-scale storage and sequential processing, it is not ideal for complex joins, aggregations, and interactive querying.

To address this, the architecture introduces a relational analytical warehouse based on PostgreSQL. A subsequent batch-oriented job (Job 4) loads integrated, analytics-ready datasets into the warehouse, which brings us to [Figure 2.6](#). This transition marks a shift from large-scale storage to query-optimized storage. The warehouse provides a structured environment where joins, aggregations, and indexing can be efficiently executed, supporting analytical and BI workloads.

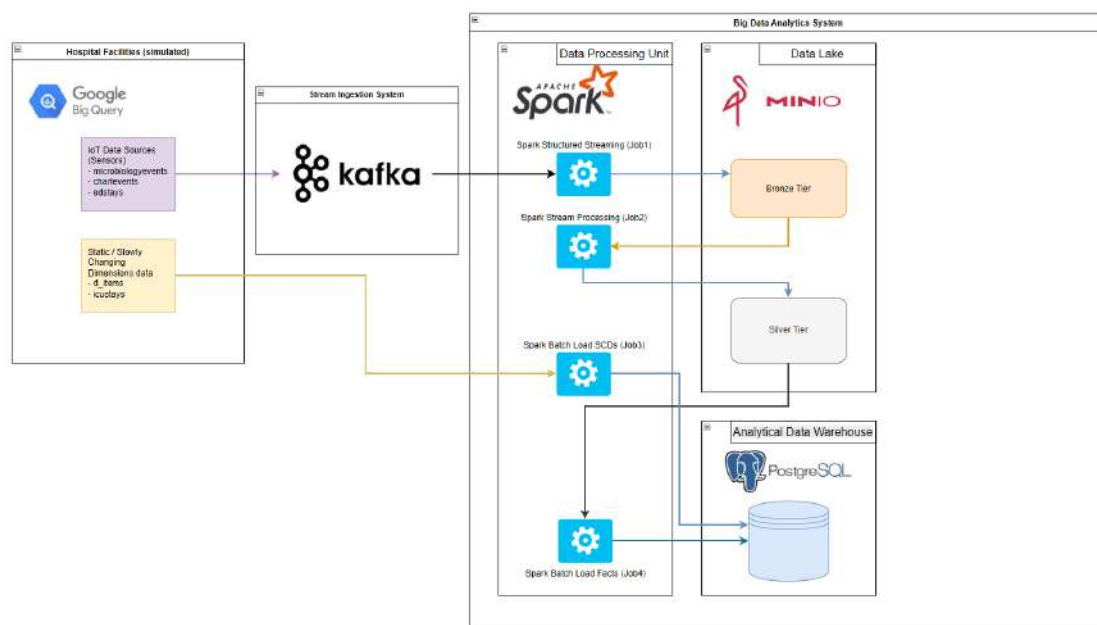


Figure 2.6: System architecture design (5)

Analytics

The final stage of the architectural diagram introduces the analytics and visualization layer. Tools such as Tableau, Streamlit, and Python-based analytics modules consume data from the PostgreSQL warehouse. These components represent the end users of the pipeline, where processed data is transformed into insights, dashboards, and interactive analyses. At this stage, the architecture demonstrates the full transformation of raw clinical data into analytics-ready information, the full architecture presentation is included in [Figure 2.7](#).

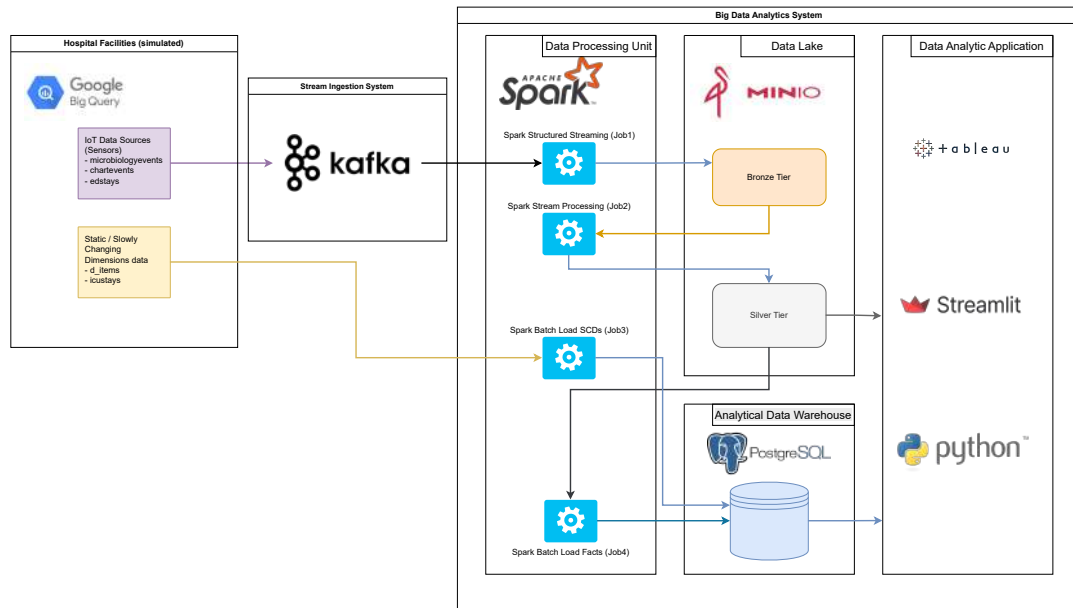


Figure 2.7: System architecture design (complete)

Overall, this gradual construction of the architectural diagram mirrors the logical evolution of the system itself. Each added component addresses a specific limitation of the previous stage, resulting in a layered pipeline that cleanly separates ingestion, processing, storage, and analytics. Presenting the architecture in this incremental manner reinforces the rationale behind each design choice and clarifies how the system supports both high-velocity streaming data and large-volume analytical processing within practical constraints.

Chapter 3

Implementation

This chapter presents the practical realization of the proposed big data pipeline, focusing on how the system was configured, integrated, and executed. Rather than emphasizing performance optimization or large-scale deployment, the implementation highlights the setup process, component interaction, and basic operational behavior of big data streaming and processing technologies. Each stage of the pipeline is described in terms of configuration choices, data flow, and observed behavior. The chapter also documents key implementation decisions made to accommodate limited local computing resources, providing context for the scope and limitations of the implemented solution and preparing the ground for subsequent discussion of results and observations.

3.1 Implementation Environment and Tooling

The implementation environment for this project is built using Docker, providing an isolated and consistent runtime for all system components. Containerization allows the full big data pipeline to be deployed and executed locally while maintaining a structure similar to real-world multi-service data platforms. [Table 3.1](#) provides an overview of the implementation environment from the view of docker containers.

Table 3.1: Containerized implementation environment overview

Container Name	Core Technology	Primary Purpose in the System	Services / Interfaces Provided
dev	Custom Docker image (Python-based)	Development and control environment for running scripts, producers, and auxiliary tasks	Interactive shell, Kafka Producer, Streamlit application (port 8501)
cp-zookeeper	Apache ZooKeeper	Coordination service required by Kafka for broker metadata management and leader election	ZooKeeper service (ports 2181, 2888)
cp-kafka	Apache Kafka	High-velocity data ingestion and buffering layer for streaming MIMIC-IV data	Kafka broker service (port 9092)
spark	Apache Spark (Structured Streaming & Batch)	Core processing engine for streaming ingestion, transformation, and batch integration jobs	Spark driver, Spark Web UI (port 4040)
minio	MinIO (S3-compatible object storage)	Data lake storage for Bronze and Silver data tiers	S3 API (port 9000), Web console (port 9001)
postgres	PostgreSQL	Analytical warehouse for curated, query-optimized datasets	PostgreSQL database service (port 5432)
pgadmin	pgAdmin 4	Web-based administration and inspection tool for PostgreSQL	pgAdmin web UI (port 8080)

At a high level, the environment includes containers for Kafka and its coordination service, Spark for both batch and streaming jobs, object storage for the data lake, and a relational database for the analytical warehouse. Supporting services, such as a database administration interface and a development container, are also included to facilitate inspection and interaction during implementation.

3.2 Data Retrieval from Google BigQuery

Data retrieval marks the first concrete implementation step of the pipeline. Rather than extracting the entire MIMIC-IV dataset, the project selectively queries a curated subset of tables that are directly relevant to downstream analytics and that reflect different data roles in a healthcare data model. This approach balances realism with feasibility, given local storage and processing constraints.

The selected tables are grouped conceptually into fact, dimension, and factless tables, as shown in [Table 3.2](#). Fact tables capture high-frequency or event-driven data and are later used to simulate high-velocity streams. Dimension tables provide descriptive and contextual attributes that evolve slowly over time. Factless tables are technically dimension tables but their facts are not in scope of the project, but they are important for some analytical tasks.

Table 3.2: Selected MIMIC-IV schemas and tables used in the system

MIMIC-IV Schema	Table Name	Data Role	Description
icu	chartevents	Fact	High-frequency ICU measurements and patient observations
icu	d_items	Dimension	Metadata describing charted measurement items
hosp	microbiologyevents	Fact	Laboratory microbiology test result events
hosp	d_icd_diagnoses join diagnoses_icd	Dimension	Diagnosis code definitions and descriptions
hosp	admissions	Dimension	Patient admission-level administrative information
icu	icustays	Factless	ICU stay episodes and associated temporal boundaries
ed	edstays	Factless	Emergency department visit and stay records
ed	triage	Dimension	Triage-level patient attributes and acuity indicators

These tables were chosen to support meaningful analytics while also enabling a clear separation between high-velocity data and slowly changing contextual data. Tables such as `chartevents` and `microbiolgyevents` are particularly important for simulating streaming workloads, as they contain large numbers of time-stamped records that resemble real-time clinical events. In contrast, tables such as `admissions`, `d_items`, and `d_icd_diagnoses` provide stable reference information used to enrich and contextualize fact data during later processing stages.

The retrieved data serves two distinct purposes in the pipeline. High-frequency fact data is exported in a form suitable for replay into Kafka, enabling controlled velocity simulation. Dimension and factless tables are retrieved through batch queries and stored locally or staged for later integration during batch processing. This dual-use strategy aligns with the architectural separation between streaming and batch workflows described earlier.

3.3 Data Ingestion with Apache Kafka

This section describes how Apache Kafka is used to ingest data into the system and to simulate high-velocity data flows using historical MIMIC-IV records. The goal is not to reproduce true real-time hospital data rates, but to study how a streaming ingestion layer behaves under continuous input and how it interacts with downstream processing components.

Each source table maps to a dedicated Kafka topic. And as mentioned earlier, only high-frequency event tables such as `icu.chartevents` and `hosp.microbiolgyevents`, are streamed through their own topics, while factless and dimension tables are handled independently.

To simulate data velocity, Kafka producers replay historical MIMIC-IV data as ordered streams. Rather than sending entire tables in a single batch, data is divided into chunks and published incrementally. The sending rate can be controlled at the producer level, allowing the system to emulate slow, moderate, or bursty ingestion patterns. This approach makes it possible to observe ingestion backpressure, consumer lag, and system stability without requiring live data sources.

From an architectural perspective, Kafka is used strictly as a transient ingestion and buffering layer. Messages are retained only long enough for consumers to process them, and Kafka is not treated as a long-term data store. This reinforces the separation of concerns: Kafka absorbs velocity and decouples producers from consumers, while durability and long-term retention are handled by the data lake.

Practical constraints play an important role in shaping the ingestion design. Given limited local storage and compute capacity, the total number of streamed records is capped, typically at around one million rows per high-velocity table. This cap is sufficient to validate ingestion logic, schema handling, and downstream processing behavior, while avoiding resource exhaustion. The emphasis is therefore on pipeline correctness and behavior, rather than on maximum throughput benchmarking.

3.4 Stream and Batch Processing with Apache Spark

Apache Spark is used as the central processing engine in the system, supporting both streaming and batch workloads. The processing logic is intentionally split into multiple Spark jobs, each with a clearly defined responsibility.

At a high level, Spark jobs are divided into streaming jobs, which react to continuous data arrival from Kafka, and batch jobs, which operate on persisted data stored in the data lake or retrieved from external sources. Although they differ in execution mode, all jobs use the same Spark-based processing model.

3.4.1 Job 1: Kafka to Bronze Data Lake

The first Spark job is a Structured Streaming job responsible for ingesting data from Kafka and persisting it to the Bronze tier of the data lake. This job subscribes to one or more Kafka topics, each corresponding to a specific high-velocity fact table.

The primary objective of this job is durability rather than transformation. Incoming messages are parsed, and written to object storage with their original structure preserved. Ordering within Kafka partitions is maintained, and records

are appended to storage in a sequential manner. Key responsibilities of this job include:

- Subscribing to Kafka topics and consuming data continuously
- Parsing message payloads into structured Spark DataFrames
- Applying basic schema enforcement and type casting
- Writing raw records to the Bronze layer in append mode

This job ensures that streamed data is no longer transient and can be replayed or reprocessed if necessary. It forms the foundation for all downstream processing.

3.4.2 Job 2: Bronze to Silver Transformation

The second Spark job focuses on transforming raw data from the Bronze layer into cleaner, standardized datasets stored in the Silver tier. Although it operates on streaming outputs, this job can be implemented either as a continuous streaming job reading incremental Bronze data or as a triggered batch-style job over accumulated Bronze files.

This job introduces meaningful preprocessing, while still avoiding heavy aggregation or complex joins. Its purpose is to prepare data for broad analytical use without over-specializing it for any single query pattern. Responsibilities of this job include:

- Parsing nested or semi-structured fields
- Handling fatal missing or malformed values
- Normalizing data types and timestamps
- Removing obvious duplicates or corrupted records

The resulting Silver datasets are still large and detailed, but significantly more consistent and reliable than raw Bronze data. They represent the main fact-level storage used for further integration.

3.4.3 Job 3: Load SCDs tables

The third Spark job is a batch-oriented integration job. It enriches the data warehouse using dimension and factless tables retrieved from the source system.

This job reflects a key architectural decision: high-velocity facts and slowly changing dimensions are handled differently. Fact data arrives continuously and is processed incrementally, while dimension data is loaded in batches and treated as relatively stable over time. Responsibilities of this job include:

- Loading dimension and factless tables (e.g., admissions, ICU stays) into data warehouse.
- Aligning records based on time windows or stay identifiers.

3.4.4 Job 4: Load Facts into Analytical Warehouse

The final Spark job prepares data from fact tables for consumption by analytics and BI tools by loading it into the PostgreSQL analytical warehouse. This job operates purely in batch mode and targets analytics-ready datasets, rather than raw or semi-processed data. Before loading, data volume is typically go through these operations:

- Transformation to comply with the data warehouse's schema.
- Reduced row via a sampling technique.
- Basic aggregations where appropriate

3.5 Data Lake Storage with MinIO

The data lake serves as the persistent storage backbone of the system, providing durable and scalable storage for data produced by streaming and batch processing jobs. In this project, the data lake is implemented using MinIO, an S3-compatible object storage system. The design follows a tiered data lake model, allowing data to evolve from raw ingestion to curated, analytics-ready form while preserving traceability.

MinIO is used purely as an object store and not as a query engine. All reading, writing, and transformation logic is handled by Spark jobs, while MinIO provides reliable storage and a standardized access interface.

3.5.1 Data Lake Design Principles

The data lake is organized according to three key principles: durability, traceability, and progressive refinement. All ingested data is written to storage as early as possible to prevent data loss. Data is never modified in place; instead, new versions are written to downstream tiers (Here we applies two tiers, as discussed in [Figure 2.7](#)). This approach ensures that earlier representations of the data remain available for inspection, debugging, or reprocessing.

Object storage is particularly suitable for this design because it supports large, immutable files and parallel access patterns. Data is stored in a format compatible with distributed processing engines.

3.5.2 Bronze Layer: Raw Streamed Data

The Bronze layer is the first persistence point for streamed data entering the system. Data written to this layer originates directly from Kafka via the initial Spark streaming job. Characteristics of the Bronze layer include:

- Minimal transformation of incoming records
- Preservation of original structure and schema
- Append-only writes reflecting arrival order

The Bronze layer acts as a safety net. If downstream processing logic changes or errors are discovered, data can be reprocessed from this layer without requiring data to be re-ingested from Kafka. It also provides a clear audit trail from ingestion to analytics.

In this project, Bronze data is written incrementally and capped at a manageable size due to local storage limitations. Despite this cap, the structure and behavior of the Bronze layer closely mirror those of large-scale production data lakes.

3.5.3 Silver Layer: Cleaned and Integrated Fact Data

The Silver layer stores cleaned, standardized, and structured datasets derived from the Bronze layer. Data in this tier is produced by subsequent Spark jobs that apply parsing, validation, and normalization logic. Compared to the Bronze layer, Silver data:

- Has consistent schemas and data types
- Excludes clearly invalid or corrupted records
- Is suitable for broad analytical use

The Silver layer is intentionally designed to remain abundant and detailed. It does not yet include heavy aggregations or application-specific logic. Instead, it represents a high-quality fact store that can support multiple downstream use cases, including warehouse loading and exploratory analysis.

3.6 Analytical Warehouse Implementation with PostgreSQL

The analytical warehouse represents the final structured storage layer in the pipeline, where processed data is organized into a relational form suitable for analytical querying and business intelligence. In this project, PostgreSQL is used to host the analytical schema that consolidates cleaned fact data and contextual dimension data produced by upstream Spark jobs.

Unlike the data lake, which emphasizes scale and flexibility, the analytical warehouse emphasizes structure, clarity, and queryability. Only curated datasets that are ready for analysis are loaded into this layer. As a result, the warehouse contains a reduced and well-defined subset of the overall data, derived primarily from the Silver tier of the data lake and enriched with dimension information.

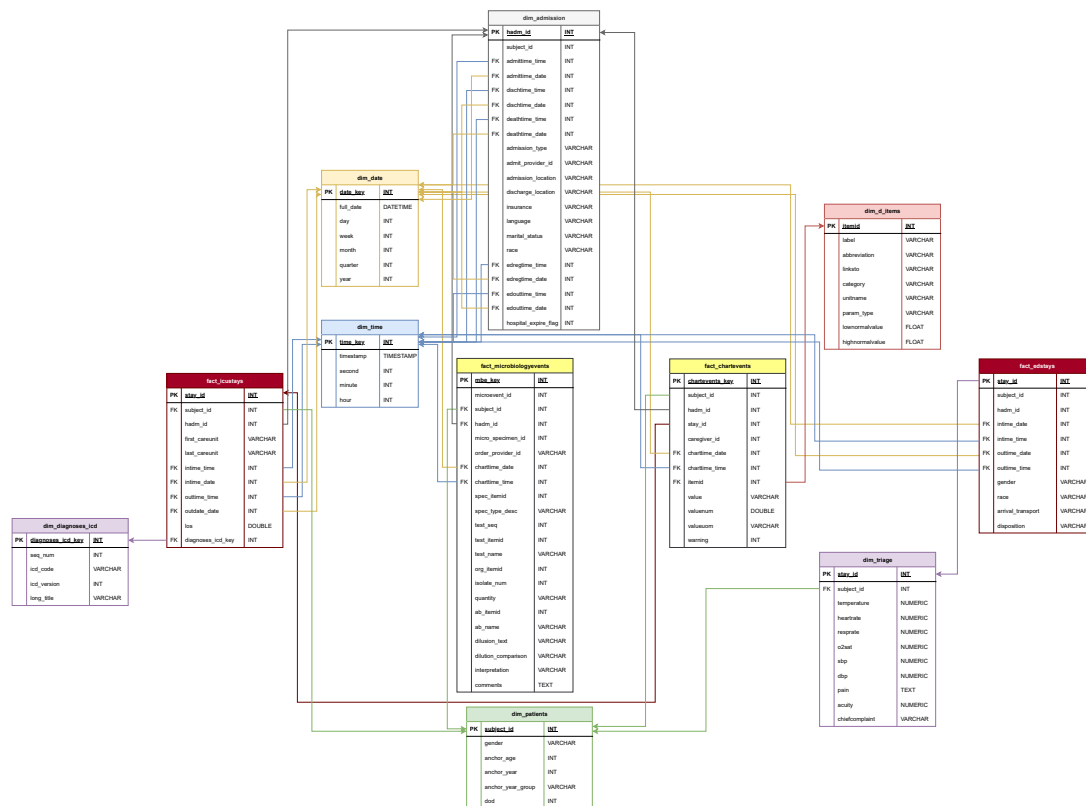


Figure 3.1: Data warehouse schema diagram

The warehouse schema in [Figure 3.1](#) is designed to support descriptive and exploratory analytics rather than transactional workloads. Tables are organized to reflect common healthcare analytical patterns, where event-level facts are linked to patient, admission, and stay contexts.

- Fact tables store event-level or measurement-level data generated at high frequency: **chartevents** - clinical observations, **microbiologyevents** - laboratory results. These tables typically contain timestamps, foreign keys to contextual entities, and measured values.
- Factless tables represent clinical episodes or stays: **icustays** for ICU stays, **edstays** for ED visits. These tables are primarily used to relate facts to time windows, locations, or care contexts.
- Dimension tables store descriptive and contextual attributes that provide meaning to fact records, such as diagnoses, triage information, or item defini-

tions. Dimension tables change slowly over time and are used to filter, group, and interpret fact data during analysis.

- Date and time dimensions (`dim_date` and `dim_time`) provide standardized representations of calendar dates and clock times, enabling consistent temporal aggregation and analysis across fact tables.

At a conceptual level, the schema follows a fact–dimension modeling approach. High-velocity event data, such as clinical measurements or laboratory events, is stored in fact tables, while descriptive attributes, such as patient demographics, diagnoses, and triage information, are stored in dimension tables. Factless tables are used to represent care episodes or stays that define analytical boundaries without contributing numerical measures.

3.7 Analytics and Visualization Layer

3.7.1 ICU Overview

The overview module provides a high-level descriptive summary of ICU activity, serving as an entry point for downstream analytical exploration. The objective of this module is to give users an immediate understanding of ICU utilization patterns across time and care units, without requiring detailed filtering or complex queries.

ICU Stays by Admission Year

The first visualization presents the distribution of ICU stays aggregated by admission year. Admission timestamps are truncated to the year level, reflecting the de-identification and date-shifting policies applied in the MIMIC dataset. Each bar represents the total number of ICU stays in a given year, where counts are computed based on unique `stay_id` values to avoid duplication across transfers or events.

This chart enables users to quickly assess temporal trends in ICU admissions, identify periods of increased or decreased activity, and contextualize subsequent analyses within broader longitudinal patterns.

ICU Stays by Care Unit

The second visualization shows the distribution of ICU stays by care unit, using the `first_careunit` attribute associated with each ICU stay. This field reflects the initial ICU unit to which a patient was admitted and does not require further semantic specification.

Counts are again calculated using unique `stay_id` values to ensure consistency with admission-level analysis. The bar chart is sorted in descending order by stay count, allowing the most frequently utilized care units to be immediately identifiable.

Together, these two bar charts provide a concise overview of ICU workload distribution across both time and organizational units. They establish essential context for more detailed trend, correlation, and risk-oriented analyses presented in subsequent modules.

3.7.2 Trend Analysis

The trend analysis module focuses on identifying dominant clinical and demographic patterns within ICU admissions. Rather than examining fine-grained temporal changes, this module highlights high-level distributions that reveal common disease burdens and patient composition in the ICU population.

Top ICD Codes by ICU Stays

The first visualization presents the top ten diagnosis codes ranked by the number of ICU stays associated with each code. ICU stays are aggregated using unique `stay_id` values to ensure that each admission is counted once, even when multiple clinical events are recorded during the same stay.

Diagnosis codes are grouped at the ICD level and sorted in descending order based on their corresponding ICU stay counts. The resulting bar chart highlights the most prevalent clinical conditions observed among ICU admissions, providing insight into dominant disease patterns that drive ICU utilization.

This visualization is particularly useful for contextualizing downstream analyses, as it identifies conditions that contribute most significantly to ICU workload and may warrant closer monitoring or targeted analytical focus.

ICU Stays by Gender

The second visualization illustrates the distribution of ICU stays by patient gender. ICU admissions are grouped into male and female categories, with the total number of stays again calculated using unique `stay_id` values.

The bar chart provides a straightforward comparison of gender representation within the ICU population. By presenting this information at an aggregate level, the visualization supports rapid assessment of demographic balance and serves as a baseline for more detailed stratified analyses in later modules.

Together, these charts offer a concise view of both clinical and demographic trends in ICU admissions, bridging high-level overview statistics with more targeted analytical investigations.

3.7.3 Correlation Analysis

The correlation analysis module explores potential relationships between physiological measurements and clinical severity indicators using visual inspection. The objective of this module is not to establish statistical significance, but to provide intuitive, visual cues that support hypothesis generation and clinical reasoning.

Scatter plots are used as the primary visualization technique, with linear regression trend lines overlaid to highlight potential linear relationships. No formal statistical metrics (e.g., correlation coefficients, confidence intervals, or p -values) are computed at this stage; interpretation is therefore qualitative rather than inferential.

SpO₂ vs. Heart Rate in ICU

The first visualization examines the relationship between average heart rate and average oxygen saturation (SpO₂) during ICU stays. Both variables are aggregated at a consistent temporal level within the same ICU stay to reduce short-term noise

and improve interpretability.

Each point in the scatter plot represents an aggregated observation, with heart rate on the horizontal axis and SpO₂ on the vertical axis. A linear regression trend line is overlaid to visually indicate whether a monotonic linear tendency may exist between the two variables.

This visualization is intended to assist in identifying broad physiological patterns, such as whether elevated heart rate tends to co-occur with reduced oxygen saturation in critically ill patients. However, no causal or statistically validated conclusions are drawn from this plot alone.

Triage Acuity vs. Heart Rate in the Emergency Department

The second visualization explores the relationship between triage acuity and heart rate in the emergency department. Triage acuity is treated as an ordinal severity indicator, where higher values correspond to more severe patient conditions.

The scatter plot displays heart rate as a function of triage acuity, with a linear regression trend line added to highlight potential directional relationships. This visualization allows users to visually assess whether higher clinical severity at triage is associated with elevated heart rate levels.

As with the ICU correlation analysis, this module serves a purely exploratory purpose. It supports visual judgment and pattern recognition but does not compute statistical significance or validate clinical hypotheses. Any observed trends are intended to motivate further quantitative or predictive analysis rather than to serve as definitive clinical evidence.

3.7.4 Sepsis vs. Non-Sepsis Analysis in the Emergency Department

This module compares ICU stays classified as sepsis and non-sepsis to highlight differences in patient distribution and physiological characteristics. The analysis is descriptive and visualization-driven, focusing on intuitive comparison rather than statistical inference.

Sepsis vs. Non-Sepsis Patient Distribution

The first visualization presents a simple two-column bar chart comparing the number of ICU stays between sepsis and non-sepsis cohorts. ICU admissions are counted using unique `stay_id` values to ensure each stay is represented only once.

This chart provides a clear overview of cohort size imbalance and establishes context for subsequent physiological comparisons. By limiting the visualization to two categories, the comparison remains immediately interpretable and avoids unnecessary diagnostic granularity at this stage.

Average Vital Signs Comparison

Average physiological measurements for sepsis and non-sepsis patients are summarized in a tabular format. Vital signs are aggregated at the ICU stay level and reported as mean values for each cohort. The table includes the following variables:

- Temperature (°C)
- Heart Rate (bpm)
- Respiratory Rate (breaths/min)
- Oxygen Saturation (%)
- Systolic Blood Pressure (mmHg)
- Diastolic Blood Pressure (mmHg)

The tabular representation enables precise value comparison between cohorts. Sepsis patients exhibit higher average temperature, heart rate, and respiratory rate, while oxygen saturation and blood pressure values are generally lower compared to non-sepsis patients. This format allows differences to be clearly identified without relying on visual approximation.

Detailed Vital Signs Analysis

To complement the summary table, a bar chart visualization is used to display detailed vital sign comparisons between sepsis and non-sepsis patients. Each vital sign is plotted side by side for the two cohorts, enabling rapid visual assessment

of relative magnitude differences.

This visualization emphasizes contrast across physiological dimensions and helps users identify which vital signs contribute most strongly to the observed cohort differences. While the chart highlights directional trends, it does not imply causality or statistical significance and is intended solely to support exploratory interpretation.

3.7.5 Predictive Analytics Module

The Predictive Analytics module represents the transition from retrospective monitoring to proactive clinical decision support. By leveraging historical patterns within the MIMIC-IV dataset, this module forecasts critical patient outcomes—Mortality, Length of Stay (LoS), and Readmission Risk—enabling healthcare providers to optimize resource allocation and intervene early.

Data Processing and Feature Engineering

To build robust predictive models, the system queries specific high-value tables: `icu.chartevents` (vitals), `icu.icustays`, `hosp.admissions`, and `hosp.diagnoses_icd`.

Preprocessing High-Velocity Data

Direct ingestion of the 40GB `chartevents` table is computationally prohibitive. A dedicated preprocessing pipeline (`data.py`) was implemented to filter critical vital signs (Heart Rate, SpO2, BP, Respiratory Rate, Temperature) and aggregate them into stay-level statistics (Mean, Min, Max) within the first 24 hours of admission.

Advanced Feature Engineering

Beyond raw statistics, we engineered domain-specific medical features to enhance model sensitivity. Key implementations included the **Shock Index** (an indicator of hemodynamic instability) and binary flags for high-risk conditions derived from ICD codes.

```
1 # 1. Calculate Shock Index (Heart Rate / Systolic BP)
2 df['shock_index'] = df['mean_heart_rate'] / df['mean_sbp']
3
4 # 2. Generate Medical Flags from ICD Codes
5 def create_flag(df_diag, codes, name):
```

```
6     admits = df_diag[df_diag['icd_code'].str.startswith(tuple(codes))]['  
       hadm_id'].unique()  
7     return pd.DataFrame({'hadm_id': admits, name: 1})  
8  
9     # Sepsis (995.91, A41), Heart Failure (428, I50), Respiratory Failure  
       (518, J96)  
10    flag_sepsis = create_flag(diagnoses, ['9959', 'A41', 'R652'], 'has_sepsis'  
       )  
11    flag_hf = create_flag(diagnoses, ['428', 'I50'], 'has_heart_failure')
```

Listing 3.1: Feature Engineering Implementation

Model 1: ICU Length of Stay (LoS) - The Accuracy vs. Recall Trade-off

This model categorizes ICU stays into *Short* (< 3 days), *Medium* (3-7 days), and *Long* (> 7 days). We experimented with two algorithms: Random Forest and XGBoost.

Baseline: Random Forest

The initial Random Forest model achieved a higher overall accuracy of **68.20%**. However, upon inspecting the confusion matrix, the model exhibited a critical failure in identifying resource-intensive patients. The Recall for the "Long Stay" class was only **16%**, meaning the model missed 84% of the patients who would stay longest in the ICU.

Optimized: XGBoost with Class Weights

In healthcare, a **False Negative** (failing to predict a long stay) is more detrimental than a **False Positive** (preparing for a long stay that ends up short). Resource shortages for unexpected long-stay patients can lead to operational bottlenecks.

To address this, we transitioned to **XGBoost** and implemented **Cost-Sensitive Learning**. We applied custom sample weights to penalize errors on the minority "Long" class significantly more than the majority "Short" class.

```
1     # Custom weights to prioritize critical classes  
2     weights_map = {  
3         le.transform(['Short'])[0]: 1,      # Standard weight  
4         le.transform(['Medium'])[0]: 2,     # Double importance  
5         le.transform(['Long'])[0]: 3.5     # Highest importance for Long Stays  
6     }  
7     sample_weights = np.array([weights_map[label] for label in y_train])  
8  
9     model = XGBClassifier(n_estimators=300, max_depth=8, objective='multi:  
       softprob')
```



```
10 model.fit(X_train, y_train, sample_weight=sample_weights)
```

Listing 3.2: Applying Custom Weights in XGBoost

Performance Comparison

As shown in [Table 3.3](#), while the overall accuracy dropped slightly to **66.34%**, the Recall for "Long Stay" patients improved dramatically from 16% to **53%**. This trade-off aligns with the medical objective of ensuring sufficient capacity for severe cases.

Model	Overall Accuracy	Recall (Short)	Recall (Long)
Random Forest (Baseline)	68.20%	98%	16%
XGBoost (Weighted)	66.34%	83%	53%

Table 3.3: Performance trade-off: XGBoost sacrifices overall accuracy to triple the detection rate of critical Long-Stay patients.

Model 2: 30-Day Readmission Risk

Predicting readmission is notoriously difficult due to the complexity of post-discharge factors. Our initial XGBoost model yielded an accuracy of 61.40% and a Recall of 56.80%. To improve the identification of high-risk patients, we performed extensive analysis on feature impacts and threshold tuning.

Feature Analysis and Risk Factors

We engineered features related to patient history, such as `previous_admissions`, `diagnosis_count`, and `age_group`. As illustrated in [Figure 3.2](#), the model correctly identifies that patients with a history of prior admissions (frequent flyers) and longer initial LoS carry a significantly higher risk score.

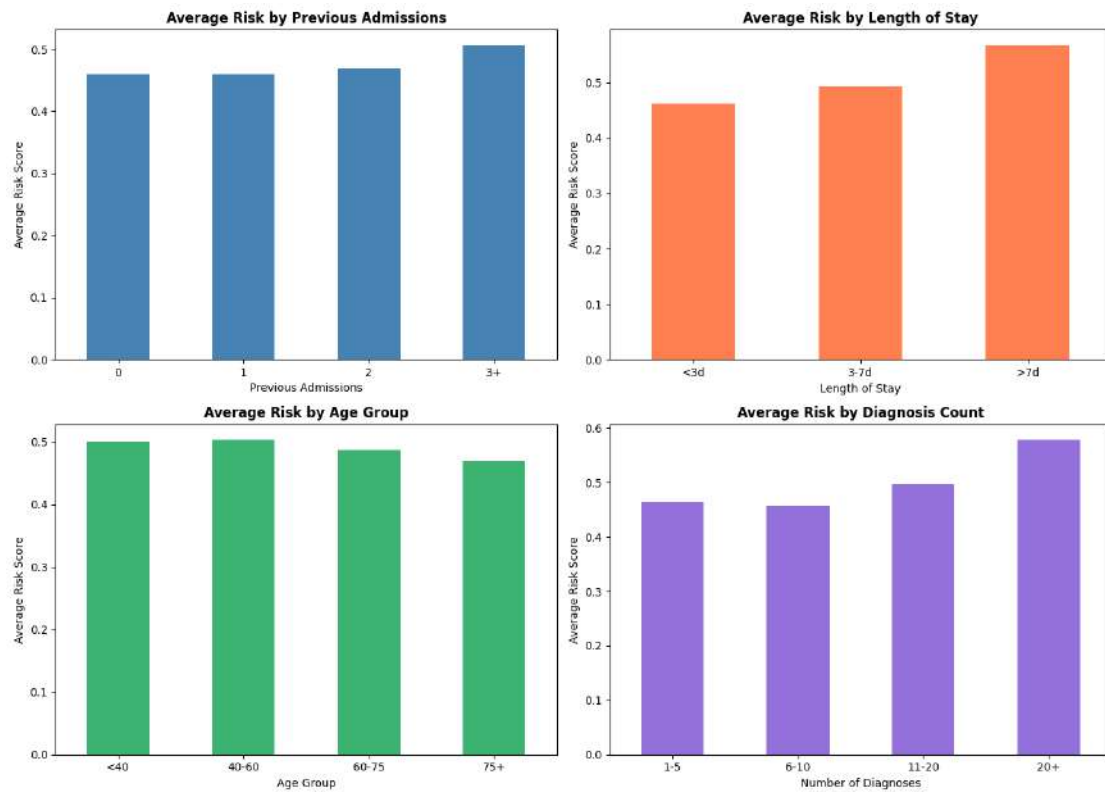


Figure 3.2: Average Readmission Risk Scores aggregated by key patient features. Patients with > 2 previous admissions and Long LoS (>7 days) show the highest risk probability.

Furthermore, we analyzed specific diagnoses associated with return visits. [Figure 3.3](#) highlights that patients with psychiatric conditions (e.g., depressive disorders) and chronic systemic issues show readmission rates exceeding 45%, providing actionable insights for discharge planning.

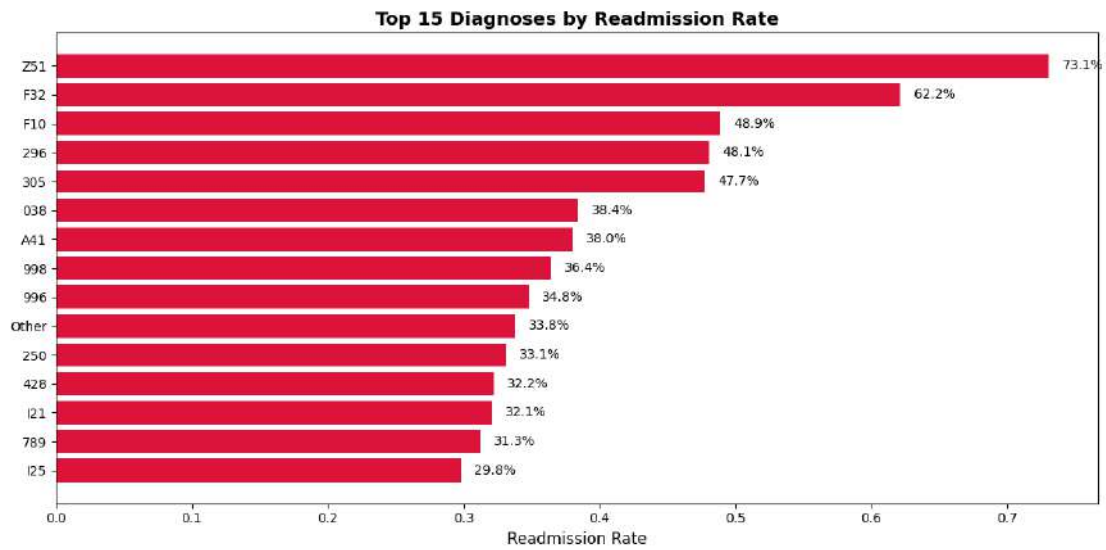


Figure 3.3: Top 15 Diagnosis Categories by Readmission Rate. High rates are observed in mental health (F32, F10) and chronic conditions.

Threshold Optimization for Sensitivity

Instead of the standard 0.5 probability threshold, we tuned the decision threshold to **0.421** based on the Precision-Recall curve analysis ([Figure 3.4](#)). This adjustment maximizes the F1-score and Sensitivity.

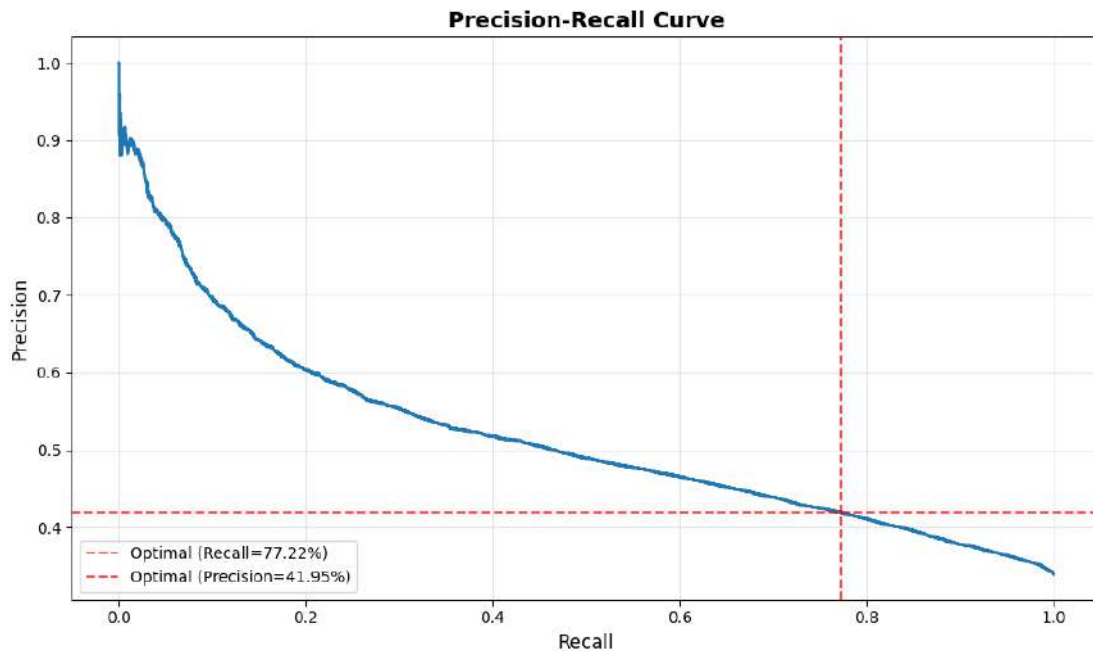


Figure 3.4: Precision-Recall Curve for Readmission Model. The optimal threshold was selected to prioritize Recall.

Final Performance:

- **Accuracy:** 56.03%
- **Recall (Sensitivity): 77.22%** (Significant improvement from baseline)

By lowering the threshold, the system successfully flags **77%** of patients likely to return, allowing case managers to schedule follow-ups for a broader group of at-risk individuals.

Model 3: In-Hospital Mortality Risk

The mortality prediction model serves as a crucial triage tool. Using the engineered features (including Shock Index and Sepsis flags), the model achieved robust performance with a **ROC-AUC of 0.871** and a **Recall of 78%**.

Feature Importance Analysis

To ensure clinical interpretability, we extracted the feature importance scores from the XGBoost model. [Table 3.4](#) displays the top predictors driving the mortality risk assessment.

Table 3.4: Top 10 Most Important Features for Mortality Prediction

Rank	Feature	Importance Score (Gain)
1	has_resp_failure	0.368 (Dominant Predictor)
2	has_shock	0.109
3	has_sepsis	0.090
4	has_renal_failure	0.078
5	has_stroke	0.043
6	has_cardiac_arrest	0.030
7	min_spo2	0.017
8	min_sbp	0.017
9	mean_resp_rate	0.016
10	age	0.015

The dominance of `has_resp_failure` (36.8%) and `has_shock` (10.9%) aligns strongly with clinical expectations, confirming that the model is learning valid physiological signals rather than spurious correlations. This interpretability fosters trust among clinical users when viewing risk scores on the dashboard.

3.7.6 Real-time Early Warning Module

While the Predictive Analytics module addresses long-term outcomes via batch processing, the Early Warning Module addresses the "Velocity" dimension of Big Data. Its primary objective is to provide situational awareness by simulating the real-time monitoring of continuous physiological streams, allowing clinicians to detect immediate patient instability.

Data Streaming Simulation

In a production ICU environment, bedside monitors generate high-frequency waveform data. To simulate this high-velocity environment using the static MIMIC-IV dataset, the system utilizes a streaming architecture backed by the pre-processed `chartevents_vitals.csv` dataset.

Rather than querying the massive 40GB raw table, the system connects to the optimized feature store created by the ETL pipeline. The application mimics a live data feed by randomly sampling active patient records upon every refresh cycle. This approach allows the dashboard to demonstrate real-time responsiveness and load handling without the latency of scanning the entire historical database.

Threshold-Based Alert Logic

Unlike the probabilistic machine learning models used in the predictive module, the Warning System operates on deterministic clinical rules. This ensures 100% explainability and immediate triggering when safety boundaries are breached.

The system monitors five core vital signs against standard medical safety thresholds. An alert status (S) is assigned to each measurement (v) based on the defined lower (T_{min}) and upper (T_{max}) limits:

$$S = \begin{cases} \text{HIGH} & \text{if } v > T_{max} \\ \text{LOW} & \text{if } v < T_{min} \\ \text{NORMAL} & \text{otherwise} \end{cases}$$

The specific thresholds configured in the system are:

- **SpO₂:** Target range 90% – 100%. Values < 90% trigger a critical Hypoxia alert.
- **Heart Rate:** Target range 60 – 100 bpm. Values > 100 indicate Tachycardia; < 60 indicate Bradycardia.
- **Systolic Blood Pressure:** Target range 90 – 140 mmHg.
- **Respiratory Rate:** Target range 12 – 20 breaths/min.
- **Temperature:** Target range 36.0°C – 38.0°C.

Visualization and Monitoring Dashboard

The interface is designed for "at-a-glance" interpretation, filtering out normal signals to focus solely on actionable alerts.

1. Aggregated KPI Metrics: The top layer of the dashboard displays distinct counters for active alerts, specifically isolating life-threatening conditions such as "Critical Hypoxia" (SpO₂ drops) and "Tachycardia Events." This immediate quantification helps the clinical team gauge the overall stress level of the unit.

2. Distribution Analysis: To understand the nature of the alerts, the system visualizes the distribution of warnings by vital sign type (as shown in [Figure 3.5](#)). This bar chart helps identify systemic issues—for example, a spike in respiratory alerts might indicate a widespread issue or equipment calibration needs.

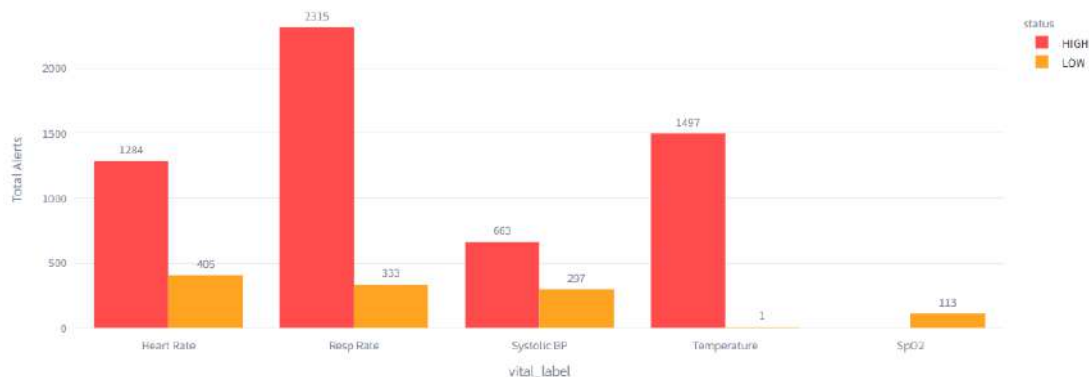


Figure 3.5: Distribution of Real-time Alerts by Vital Sign. The system aggregates high-frequency streams to highlight the most prevalent physiological instabilities.

3. Priority Patient List: The granular view provides a detailed table of patients currently triggering alerts. To facilitate rapid scanning, conditional formatting is applied: rows with **HIGH** or **LOW** status are highlighted in red. This allows clinicians to instantly identify the specific `stay_id` and the exact magnitude of the deviation, bridging the gap between aggregate data and individual patient care.

ICU Stay ID	Hours from Admit	Vital Sign	Measured Value	unit	Min Limit	Max Limit	Alert Type
37278380	12.3 h	SpO2	89.00	%	90.000000	100.000000	LOW
31676304	2.0 h	SpO2	89.00	%	90.000000	100.000000	LOW
39473453	15.2 h	SpO2	89.00	%	90.000000	100.000000	LOW
33813225	15.3 h	Systolic BP	89.00	mmHg	90.000000	140.000000	LOW
52484838	0.0 h	Systolic BP	89.00	mmHg	90.000000	140.000000	LOW
36907946	42.8 h	Systolic BP	89.00	mmHg	90.000000	140.000000	LOW
38360173	15.7 h	Systolic BP	89.00	mmHg	90.000000	140.000000	LOW
35031967	47.9 h	Systolic BP	89.00	mmHg	90.000000	140.000000	LOW
30643499	12.0 h	Systolic BP	89.00	mmHg	90.000000	140.000000	LOW
38054101	25.7 h	SpO2	89.00	%	90.000000	100.000000	LOW

Figure 3.6: Patient List

Chapter 4

Results and Discussion

This chapter presents the results obtained from executing the proposed big data pipeline and discusses the insights derived from both the system behavior and the analytical outcomes. The discussion is divided into two parts: the first examines the end-to-end data pipeline, from ingestion to the analytical warehouse, while the second focuses on analytical results and insights from the data.

4.1 Big Data Pipeline

This section reports and discusses the observed behavior of the data pipeline when executed end-to-end. The flow in this section follows the flow of data presented in [subsection 2.2.2](#). The focus is on validating system behavior, examining how components interact in practice, and reflecting on strengths and limitations revealed during execution.

4.1.1 Streaming Ingestion and Bronze Layer

The first operational stage demonstrates streaming ingestion using Apache Kafka. A Kafka producer is executed, spawning parallel processes that stream historical MIMIC-IV records into the system. Although the data originates from static files, it is emitted incrementally to simulate high-velocity event streams.

The first active stage of the pipeline demonstrates data ingestion via Apache Kafka. A Kafka producer is executed, spawning parallel processes that stream historical MIMIC-IV data into Kafka topics. Then the first Spark streaming job (presented in [subsection 3.4.1](#)) is started, subscribing to these topics and writing

incoming records to the Bronze tier of the data lake. [Figure 4.1](#) shows the bronze tier bucket in the data lake when the pipeline has started running.

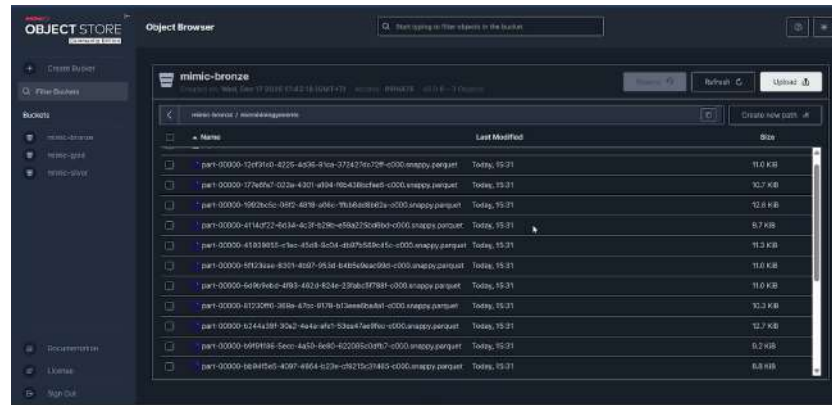


Figure 4.1: Bronze tier data lake

This stage demonstrates the effectiveness of Kafka as a velocity-handling layer and validates the decoupling between ingestion and storage. A key strength observed here is that ingestion can proceed independently of downstream processing speed. However, the demonstration also exposes practical constraints: ingestion rates and total volume are intentionally capped to avoid exhausting local resources. As a result, while functional correctness is validated, throughput and scalability characteristics cannot be meaningfully evaluated.

4.1.2 Silver Layer Processing

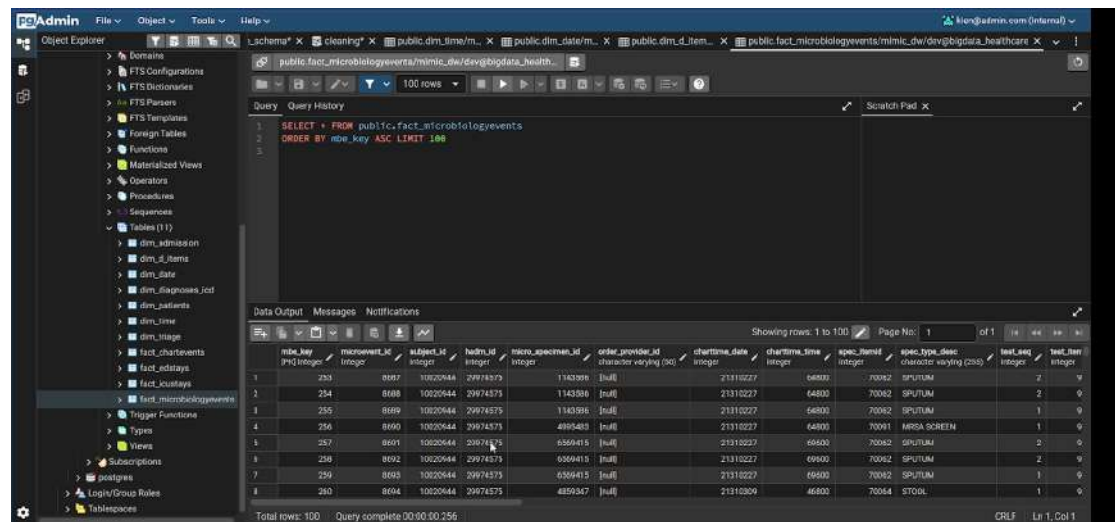
The second Spark job (presented in [subsection 3.4.2](#)) is then executed to process data from the Bronze tier and write refined output to the Silver tier. This job is set to stream processing the data from the bronze-tier layer to the silver-tier layer in mini-batches of 1 minutes of data. This processing includes parsing, basic cleaning, and the addition of a `processed_timestamp` column that records when each record is written to the Silver layer. This attributes will be used later as a watermark for the import of data into the warehouse in later steps.

This stage illustrates the value of a layered data lake design. By separating raw and refined data, the pipeline supports traceability and reprocessing while still offering cleaner datasets for downstream use. The introduction of `processed_timestamp` proves particularly important later, as it enables incremen-

tal batch processing.

4.1.3 Batch Integration and Warehouse Population

The final demonstrated stage involves running two batch jobs (Job 3 and Job 4, presented in [subsection 3.4.3](#) and [subsection 3.4.4](#), respectively) that integrate Silver data and load it into the PostgreSQL analytical warehouse. These jobs use a watermark based on `processed_timestamp` to ensure that only previously unprocessed records are selected during each run. After execution, the warehouse tables transition from empty to populated. [Figure 4.2](#) shows the populated warehouse at the end of the pipeline.



mbe_key	microevent_id	subject_id	host_id	micro_specimen_id	order_provider_id	charttime_date	charttime_time	spec_item_id	spec_type_desc	test_sas	test_name
1	253	8087	10020444	29674575	1143596	21310227	04000	70062	SPUTUM	2	9
2	254	8088	10020444	29674575	1143596	21310227	04000	70062	SPUTUM	2	9
3	255	8089	10020444	29674575	1143596	21310227	04000	70062	SPUTUM	1	9
4	256	8090	10020444	29674575	4859483	21310227	04000	70061	MRSA SCREEN	1	9
5	257	8091	10020444	29674575	6569415	21310227	04000	70062	SPUTUM	2	9
6	258	8092	10020444	29674575	6569415	21310227	04000	70062	SPUTUM	2	9
7	259	8093	10020444	29674575	6569415	21310227	04000	70062	SPUTUM	1	9
8	260	8094	10020444	29674575	4859483	21310309	04000	70064	STOOL	1	9

Figure 4.2: Resulted analytical warehouse

This approach represents a pragmatic compromise. While the warehouse loading jobs are batch-oriented and manually triggered, the watermark mechanism allows them to behave incrementally. This design reduces complexity while still demonstrating a realistic data integration pattern. The main limitation is the lack of automation and near-real-time updates, which would be expected in production systems.

4.2 Analytical Outcomes

Key insights from MIMIC-IV, such as patient admission trends, diagnosis frequencies, predictions, etc... By showing screenshots and provides explanation.

4.2.1 Module 1: Data OverView

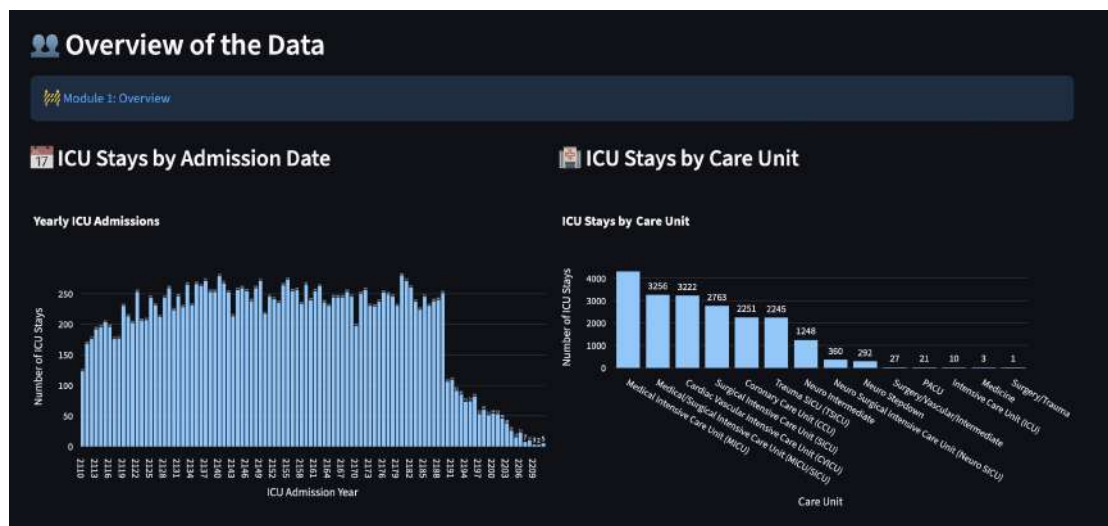


Figure 4.3: Overview of ICU utilization. The left panel illustrates ICU stays by admission date, showing temporal trends in ICU admissions over time. The right panel depicts ICU stays by care unit, highlighting the distribution of ICU utilization across different types of intensive care units.

Figure 4.3 illustrates ICU utilization from both temporal and organizational perspectives. The left panel shows the yearly number of ICU admissions over the observed period. ICU stays remain generally stable and high across most years, with admissions fluctuating within a relatively narrow range, indicating consistent utilization of ICU resources over time.

In the most recent years, a noticeable decline in ICU admissions is observed. This reduction is likely attributable to incomplete data coverage for later years rather than a genuine decrease in ICU demand, suggesting that critical care needs remain sustained throughout the dataset period.

The right panel presents the distribution of ICU stays by care unit. Medical Intensive Care Units (MICU) and combined Medical/Surgical ICUs account for the largest proportion of ICU stays, underscoring their central role in critical care delivery. Cardiac, surgical, coronary, and trauma ICUs also contribute substantially to overall ICU utilization.

In contrast, specialized units such as neurological step-down units, post-anesthesia care units (PACU), and intermediate care units record relatively few ICU stays. This uneven distribution reflects differences in patient acuity, admission criteria,

and the specialized functions of these units.

4.2.2 Module 2 : Trend Analysis

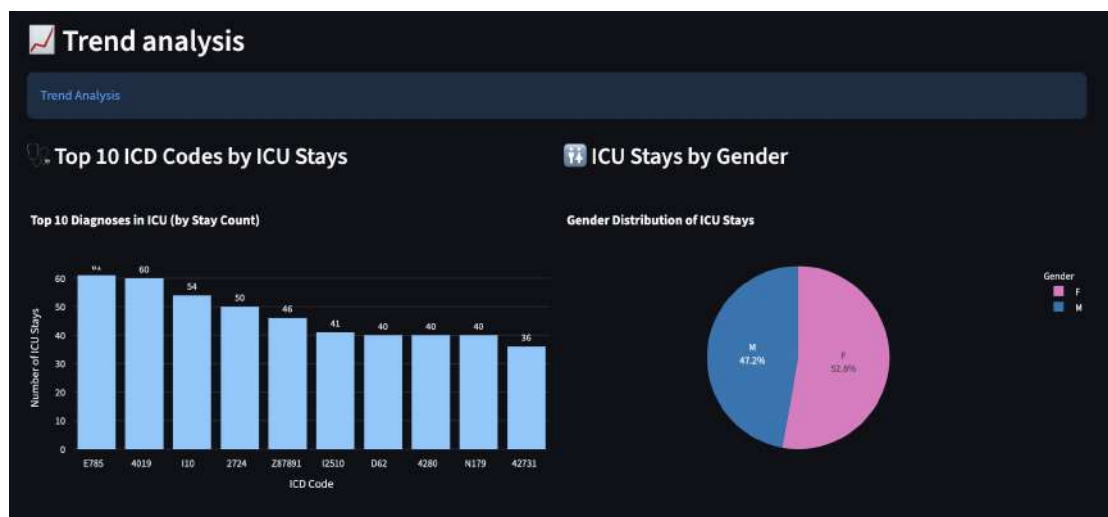


Figure 4.4: Trend analysis of ICU utilization. The left panel shows the top ten ICD diagnosis codes by ICU stay count, highlighting the most frequent conditions requiring intensive care. The right panel presents the gender distribution of ICU stays, illustrating the proportion of male and female patients admitted to the ICU.

Figure 4.4 summarizes key trends in ICU utilization based on diagnosis patterns and patient demographics. The left panel indicates that ICU admissions are concentrated among a limited set of ICD codes, with the top diagnoses accounting for the highest number of ICU stays. This concentration suggests that a small number of critical conditions contribute disproportionately to ICU workload, emphasizing the importance of targeted clinical management and preventive strategies for these diagnoses.

The right panel shows the gender distribution of ICU stays, with female patients accounting for approximately 52.8% of admissions and male patients representing 47.2%. This relatively balanced distribution indicates comparable ICU utilization across genders, with a slight predominance of female admissions. Overall, the figure highlights diagnosis-driven ICU demand alongside a broadly even demographic distribution.

4.2.3 Module 3: Correlation Analysis



Figure 4.5: Correlation analysis of physiological and clinical indicators. The left panel shows the relationship between average heart rate and average oxygen saturation (SpO₂) in ICU patients, aggregated hourly within the same ICU stay. The right panel illustrates the correlation between triage acuity level and heart rate in the emergency department, where higher acuity values indicate greater severity.

Figure 4.5 explores correlations between vital signs and clinical severity measures in both ICU and emergency department settings. The left panel demonstrates a weak negative correlation between heart rate and oxygen saturation (SpO₂) among ICU patients. As heart rate increases, a slight decrease in SpO₂ can be observed, although the overall dispersion of data points indicates substantial variability. This suggests that while elevated heart rate may be associated with reduced oxygenation, the relationship is not strongly linear and is influenced by multiple clinical factors.

The right panel presents the relationship between triage acuity and heart rate in the emergency department. A downward trend is observed, indicating that patients with higher triage acuity scores (representing more severe clinical conditions) tend to exhibit lower average heart rates. However, the wide spread of heart rate values within each acuity level highlights significant inter-patient variability, suggesting that triage severity alone does not fully explain heart rate differences.

Overall, the correlation analysis indicates that while certain physiological and

clinical indicators show directional relationships, these associations are relatively weak and complex, underscoring the multifactorial nature of patient condition assessment in critical and emergency care environments.

4.2.4 Module 4 : Sepsis vs Non-Sepsis Analysis

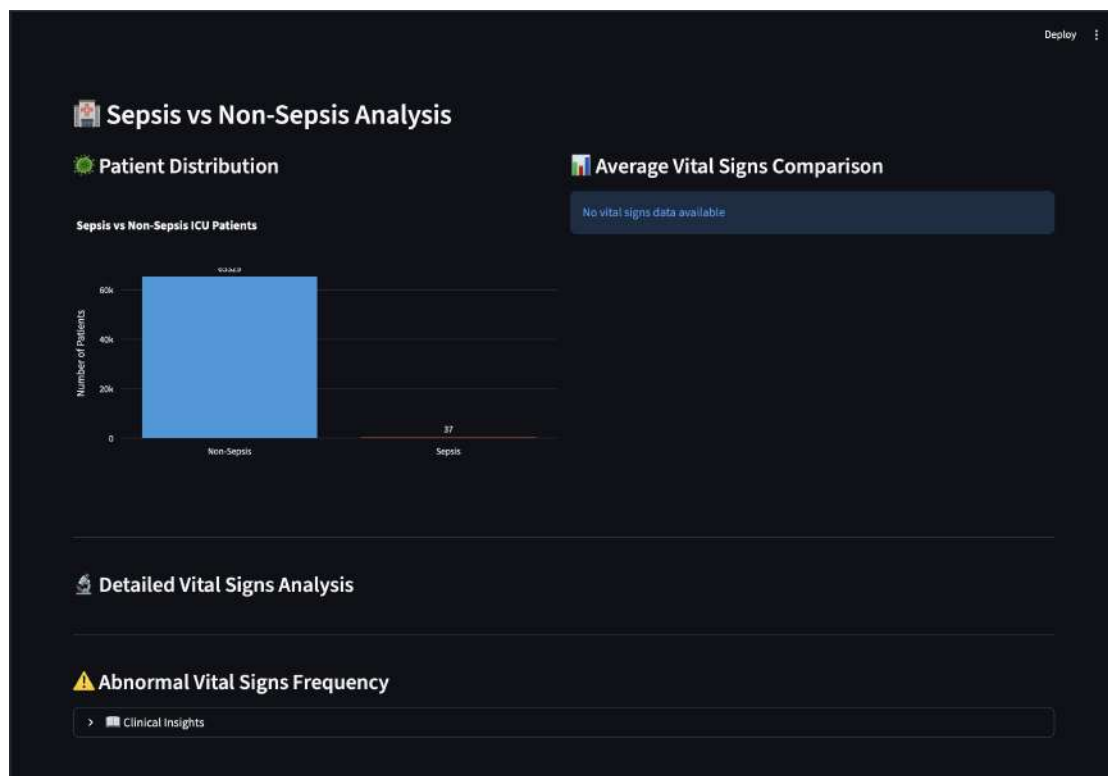


Figure 4.6: Comparison of sepsis and non-sepsis ICU patients. The figure shows the distribution of ICU patients by sepsis status, highlighting a substantial imbalance between non-sepsis and sepsis cases.

Figure 4.6 presents the distribution of ICU patients classified as sepsis and non-sepsis. The results reveal a pronounced class imbalance, with non-sepsis patients overwhelmingly dominating the dataset, while sepsis cases represent only a very small fraction of total ICU admissions.

Due to this extreme imbalance, direct comparison of average vital signs and detailed physiological patterns between sepsis and non-sepsis groups is not statistically meaningful. The limited number of sepsis cases prevents reliable estimation of central tendencies and variability, and any observed differences may be driven

by noise rather than true clinical signals.

As a result, further comparative analyses of vital signs between sepsis and non-sepsis patients are constrained by data availability. Robust comparison would require either a larger sepsis cohort, resampling techniques, or alternative analytical approaches designed to handle highly imbalanced clinical datasets. .

4.2.5 Module 5: Real-time Early Warning System

The Early Warning System is designed to address the "Velocity" aspect of Big Data, simulating a real-time monitoring environment where physiological data streams are continuously scanned for abnormalities.

Dashboard Overview

Figure 4.7 illustrates the main interface of the monitoring module. The system connects to the processed data stream (simulated via the `chartevents_vitals` dataset) and applies deterministic medical thresholds to detect patient instability.



Figure 4.7: The Real-time Early Warning Dashboard displaying active alerts and patient status.

Key Components and Insights

Based on the visualization in Figure 4.7, the analytical outcomes include:

- **Sidebar Controls (Data Filter):** Located on the left panel, this component allows clinicians to interactively filter the data stream. Users can toggle

specific alert levels (e.g., viewing only 'HIGH' priority warnings) or select specific vital signs of interest. This interactivity transforms the dashboard from a static display into an exploratory tool, enabling focused monitoring of specific patient cohorts.

- **KPI Metrics (Top Layer):** The dashboard provides immediate situational awareness through aggregated counters.

- *Active Alerts:* Indicates the total volume of abnormal events currently occurring in the ICU.
- *Critical Hypoxia (SpO_2):* Specifically isolates patients with oxygen saturation $< 90\%$. This is a life-threatening condition requiring immediate intervention.
- *Tachycardia Events:* Counts instances where heart rate exceeds 100 bpm, signaling potential cardiac stress or shock.

- **Distribution Charts (Middle Layer):**

- The *Alert Distribution* bar chart breaks down warnings by vital sign type. In the sample outcome, we often observe that Heart Rate and Blood Pressure generate the most alerts, reflecting the high volatility of hemodynamic status in ICU patients.
- The *Severity Ratio* pie chart visualizes the proportion of HIGH vs. LOW alerts, helping resource managers understand the nature of the clinical demand.

- **Priority Patient List (Bottom Layer):** The detailed table serves as a worklist for clinicians. Conditional formatting (red highlights) is applied to critical values. This allows doctors to instantly identify *which* patient (`stay_id`) is deteriorating and by *how much* (e.g., SpO_2 at 88% vs. threshold of 90%).

Implementation Review

Strengths: The module successfully reduces information overload. Instead of displaying raw waveforms for all patients, it filters and presents only actionable



exceptions. The "Refresh Data Stream" mechanism effectively simulates the velocity of incoming sensor data.

Limitations: Currently, the threshold logic is static (fixed values for all patients). A future improvement would be implementing "Personalized Thresholds" where the alarm limits adjust dynamically based on the patient's baseline condition or specific diagnosis.

4.2.6 Module 6: Predictive Clinical Decision Support

The Predictive Module focuses on individual patient assessment. Unlike the monitoring module which looks at current vitals, this module uses Machine Learning (XGBoost) to forecast future risks, specifically assessing the probability of Mortality, ICU Readmission, and Length of Stay (LoS).

Interface Overview

Figure 4.8 shows the "Single Patient Assessment" interface. This tool is designed for bedside usage, allowing clinicians to manually input physiological and demographic parameters to obtain an immediate, comprehensive risk profile.

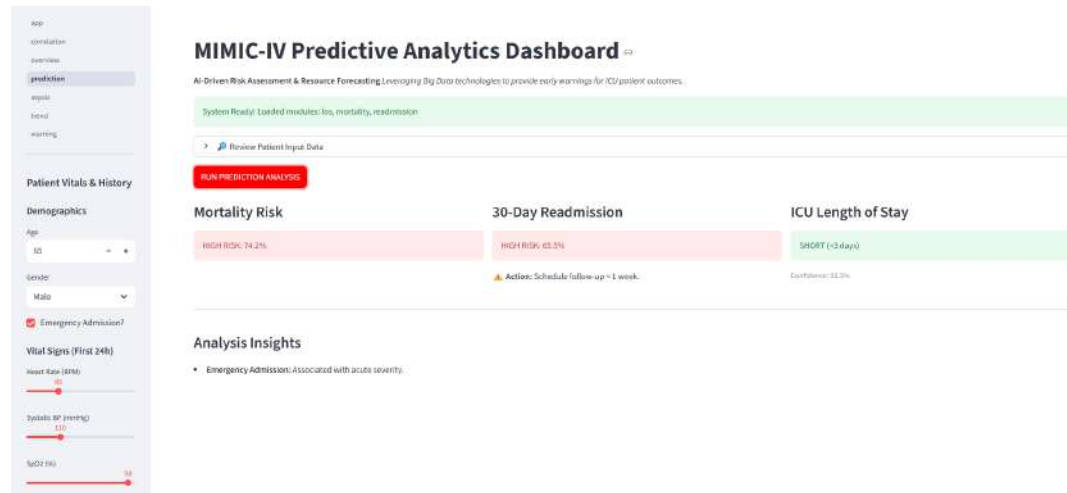


Figure 4.8: Manual Prediction Interface for assessing individual patient risk across multiple outcomes.

Key Components and Insights

The outcome of this module is interpreted through the following workflow:

- **Parameter Input:** The sidebar allows the input of key features identified during the feature engineering phase, including:
 - *Demographics:* Age and Admission Type (Emergency/Elective).
 - *Clinical History:* Number of prior diagnoses and previous admissions.
 - *Physiological Aggregates:* Mean Heart Rate, SpO₂, and Length of Stay (LoS).

- **Comprehensive Risk Assessment:** Upon processing the inputs, the dashboard runs three separate XGBoost models in parallel to provide a holistic view of the patient's prognosis:
 1. **Mortality Risk:** Estimates the likelihood of in-hospital death. High probabilities ($>50\%$) trigger a red alert, signaling the need for intensive monitoring.
 2. **30-Day Readmission:** Predicts the probability of returning to the ICU. If the risk is high ($>50\%$), the system recommends scheduling a follow-up within one week.
 3. **Length of Stay (LoS):** Classifies the expected duration into Short (<3 days), Medium, or Long (>7 days). This assists in bed management and capacity planning.
- **Automated Analysis Insights:** Beyond raw probabilities, the system provides an "Analysis Insights" section (bottom of the dashboard) to explain *why* a certain risk level was assigned. It automatically flags critical contributors such as:
 - **Multimorbidity:** High diagnosis count indicating complex disease burden.
 - **Sepsis Detection:** A major risk factor for both mortality and extended stay.
 - **Hemodynamic Instability:** Flagged when the calculated Shock Index ($HeartRate/SystolicBP$) exceeds 0.9.
- **Example Scenario Outcome:** As shown in the screenshot, for an elderly patient (e.g., >80 years) with a history of emergency admissions and high diagnosis count, the model predicts a "**High Risk**" status across multiple dimensions. This aligns with clinical intuition that multimorbidity and advanced age are strong drivers of adverse outcomes.

Implementation Review

Strengths: The manual input interface ensures that the model is accessible even when the automated data pipeline is unavailable or when assessing a new patient not yet fully integrated into the electronic health record. It serves as an effective



”Second Opinion” tool for discharge decisions.

Limitations: The primary limitation is the manual entry requirement, which is not scalable for assessing the entire ICU population simultaneously. While the system supports individual care, integrating it into the automated batch processing pipeline would significantly enhance workflow efficiency. Additionally, the model’s accuracy is dependent on the quality of inputs provided by the user.

Chapter 5

Conclusion

This project successfully designed and simulated an end-to-end big data pipeline capable of addressing the critical challenges of Volume and Velocity in healthcare environments. By utilizing the MIMIC-IV clinical dataset, the system demonstrated how modern open-source technologies can be integrated to transform raw, high-frequency medical events into actionable clinical insights.

Technologically, the project validated a layered architecture that decouples data ingestion from processing and storage. The implementation of Apache Kafka proved effective as a buffering layer for high-velocity streaming data, simulating realistic bedside monitoring environments. Apache Spark served as a unified engine, successfully handling both the continuous ingestion of raw events into the Data Lake (Bronze/Silver tiers) and the batch integration of dimensional data into the PostgreSQL analytical warehouse. This modular approach ensured data durability, traceability, and consistency across the pipeline.

However, the implementation also highlighted specific limitations. The scale of data was constrained by local hardware resources, requiring caps on total ingestion volume. Additionally, the analysis of sepsis patients revealed significant challenges related to class imbalance, which limited the statistical power of comparative vital sign analyses.

In conclusion, while implemented on a reduced scale, this project provides a functional blueprint for a healthcare big data platform. It confirms that a hybrid architecture combining streaming and batch processing can effectively manage



the complexity of clinical data, bridging the gap between raw sensor feeds and analytical insights.

Chapter 6

Appendix

6.1 Host System Specification

This appendix summarizes the hardware and execution environment used for the development and evaluation of the data pipeline. The configuration reflects a constrained, single-machine setup, which directly influences several architectural and operational decisions discussed in the main chapters.

Table 6.1: System Hardware Specification

Component	Specification
Processor	11th Gen Intel® Core™ i7-11800H @ 2.30 GHz
Installed Memory	16.0 GB RAM (15.8 GB usable)
System Type	64-bit operating system, x64-based processor

6.2 Full Project Source Code

The full source code for this project, including data ingestion scripts, stream and batch processing jobs, infrastructure configuration, and supporting utilities, is publicly available in a GitHub repository. The repository reflects the complete implementation corresponding to the pipeline and experiments discussed in this report.

The latest source code and materials can be found [here](#).

References

- [1] A. Badman and M. Kosinski, *What is big data?* Nov. 2024. [Online]. Available: <https://www.ibm.com/think/topics/big-data>.
- [2] H.-Y. Huang et al., “Hint: A comprehensive collection of human-annotated transcription factor binding sites,” *Nucleic Acids Research*, vol. 47, no. D1, pp. D267–D273, Jan. 2019. DOI: [10.1093/nar/gky1093](https://doi.org/10.1093/nar/gky1093). [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC6381367/>.
- [3] A. E. Johnson et al., *Mimic-iv, a freely accessible electronic health record dataset*, 2024. [Online]. Available: <https://physionet.org/content/mimiciv/3.1/>.
- [4] MIMIC Code Repository, *Chartevents - chartevents*, 2024. [Online]. Available: <https://mimic.mit.edu/docs/iv/modules/icu/chartevents/>.
- [5] MIMIC Code Repository, *Mimic-iv documentation home*, 2024. [Online]. Available: <https://mimic.mit.edu/docs/iv/>.
- [6] A. E. W. Johnson et al., “Mimic-iv, a freely accessible electronic health record dataset,” *Scientific Data*, vol. 10, no. 1, p. 1, Jan. 2023. DOI: [10.1038/s41597-022-01899-x](https://doi.org/10.1038/s41597-022-01899-x). [Online]. Available: <https://www.nature.com/articles/s41597-022-01899-x>.
- [7] Google, *Introduction to bigquery*, 2024. [Online]. Available: <https://cloud.google.com/bigquery/docs/introduction>.
- [8] The Apache Software Foundation, *Introduction to apache kafka*, 2024. [Online]. Available: <https://kafka.apache.org/41/getting-started/introduction/>.
- [9] The Apache Software Foundation, *Structured streaming programming guide - api on dataframes and datasets*, 2024. [Online]. Available: <https://spark.apache.org/docs/latest/streaming/apis-on-dataframes-and-datasets.html>.
- [10] Confluent, *Kafka delivery semantics*, 2024. [Online]. Available: <https://docs.confluent.io/kafka/design/delivery-semantics.html>.
- [11] MinIO, *Modern data lakes and lakehouses*, 2024. [Online]. Available: <https://www.min.io/solutions/modern-data-lakes-lakehouses>.
- [12] Hostinger, *What is postgresql?* 2024. [Online]. Available: <https://www.hostinger.com/tutorials/what-is-postgresql>.
- [13] Salesforce, *What is tableau?* 2024. [Online]. Available: <https://www.tableau.com/why-tableau/what-is-tableau>.
- [14] Streamlit, *Streamlit documentation*, 2024. [Online]. Available: <https://docs.streamlit.io/>.
- [15] Python Software Foundation, *About python*, 2024. [Online]. Available: <https://www.python.org/about/>.